

P99 CONF



A ScyllaDB Community

github.com/RagnarGrootKoerkamp/quickheap
curiouscoding.nl/posts/quickheap
ESA 2026 Best Paper

SimdQuickHeap: The fastest priority queue



Ragnar Groot Koerkamp
Postdoc High Throughput Data Structures

Joint work with Johannes Breitling and Marvin Williams



Motivation: *practical* improvements

Breaking the Sorting Barrier for Directed Single-Source Shortest Paths

Ran Duan *

Jiayi Mao *

Xiao Mao [†]

Xinkai Shu [‡]

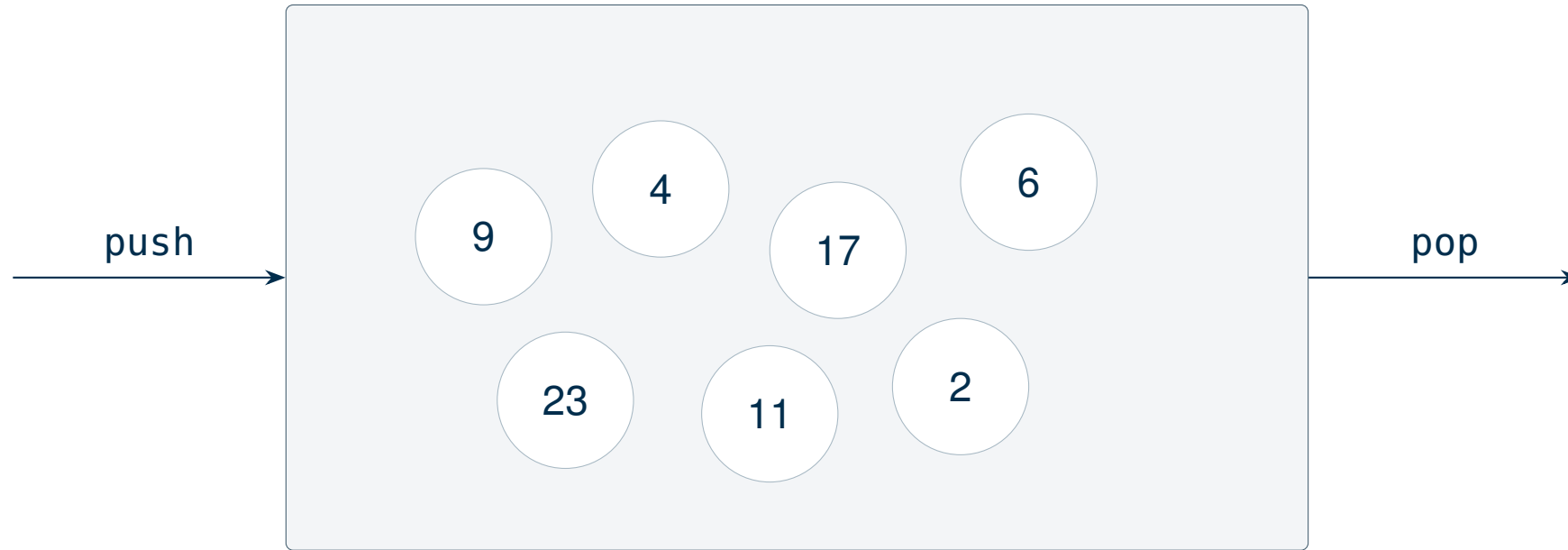
Longhui Yin *

July 31, 2025

Abstract

We give a deterministic $O(m \log^{2/3} n)$ -time algorithm for single-source shortest paths (SSSP) on directed graphs with real non-negative edge weights in the comparison-addition model. This is the first result to break the $O(m + n \log n)$ time bound of Dijkstra's algorithm on sparse graphs, showing that Dijkstra's algorithm is not optimal for SSSP.

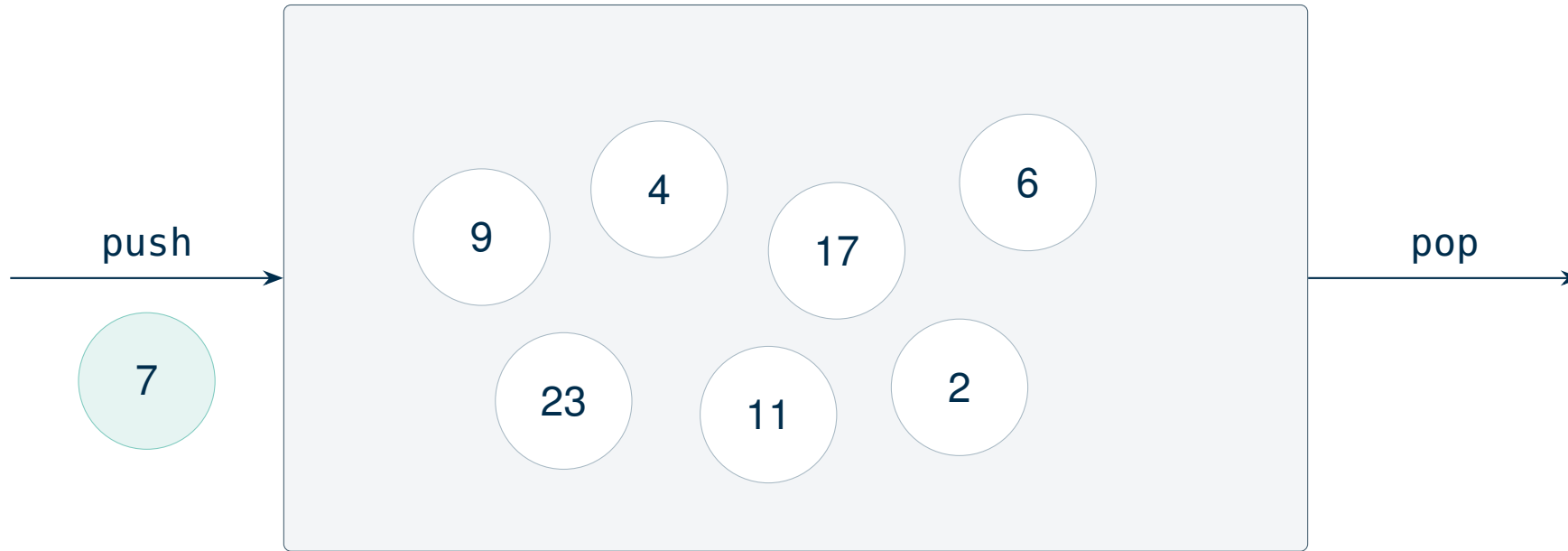
Priority Queues



Convention: **smaller** elements have **higher** priority

Applications: Dijkstra, A*, Jarník–Prim, event simulation, job scheduling, ...

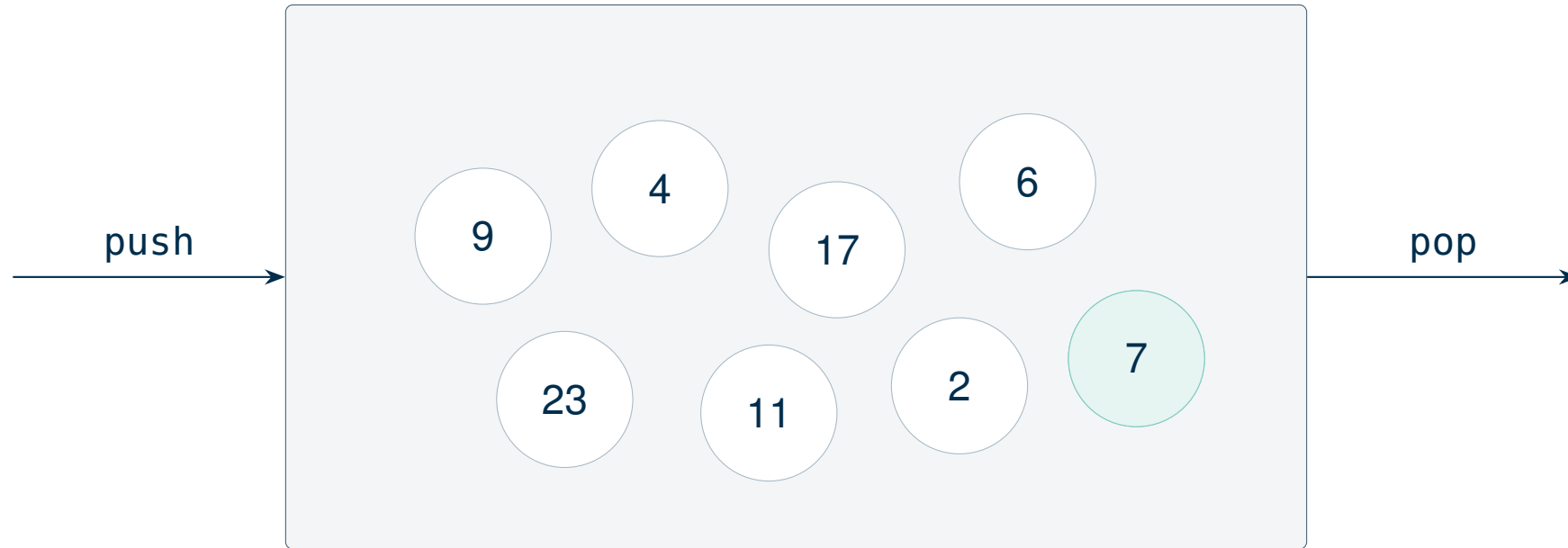
Priority Queues



Convention: **smaller** elements have **higher** priority

Applications: Dijkstra, A*, Jarník–Prim, event simulation, job scheduling, ...

Priority Queues



Convention: **smaller** elements have **higher** priority

Applications: Dijkstra, A*, Jarník–Prim, event simulation, job scheduling, ...

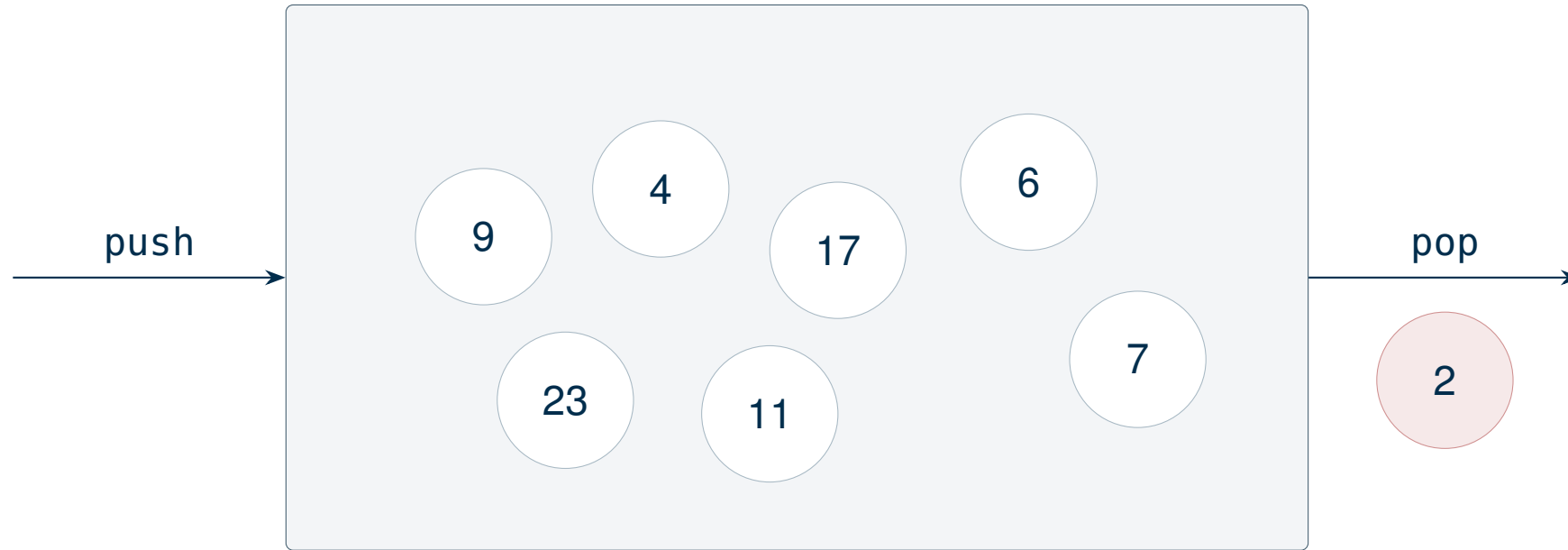
Priority Queues



Convention: **smaller** elements have **higher** priority

Applications: Dijkstra, A*, Jarník–Prim, event simulation, job scheduling, ...

Priority Queues



Convention: **smaller** elements have **higher** priority

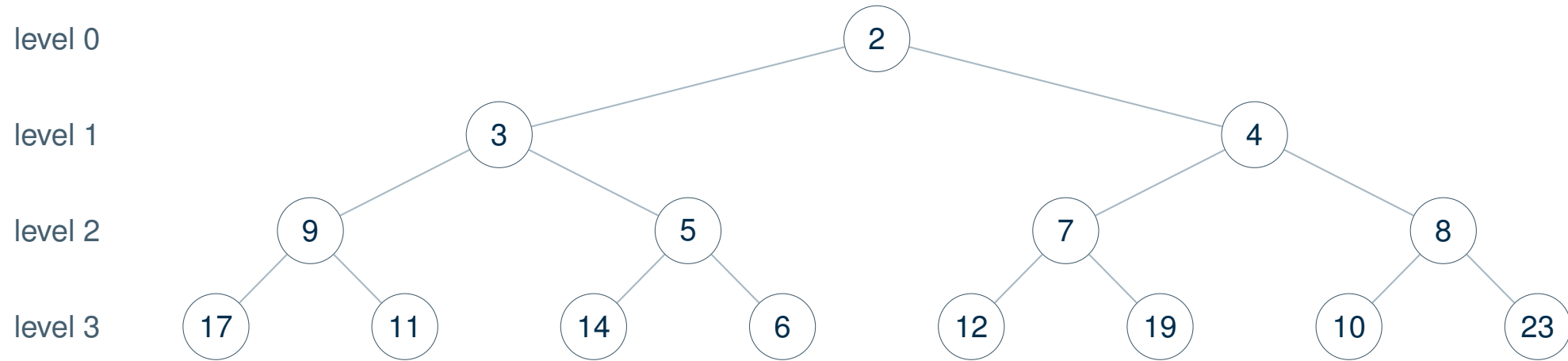
Applications: Dijkstra, A*, Jarník–Prim, event simulation, job scheduling, ...

Theory vs. Practice

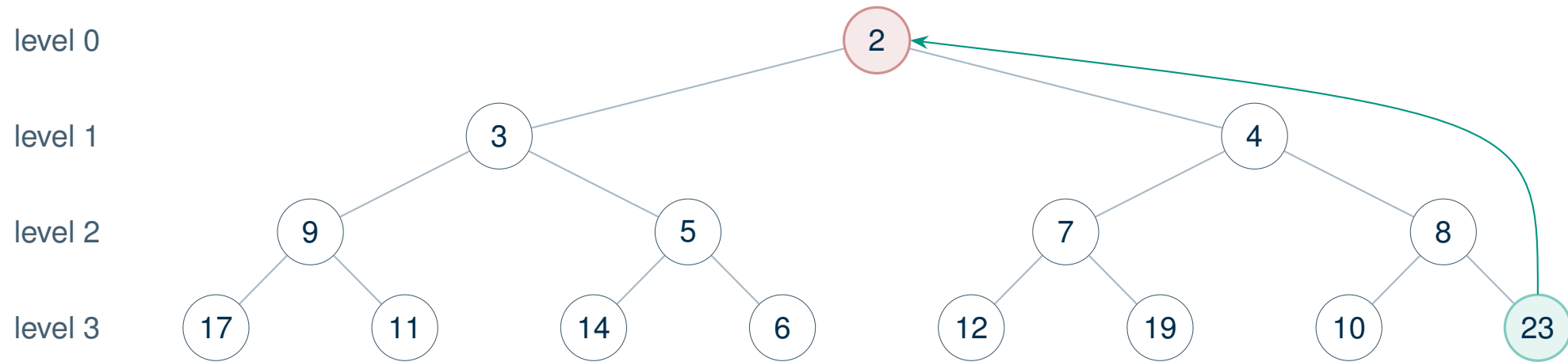
	push	pop	in practice
Binary heap	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	fast, simple, universal
Fibonacci heap	$\mathcal{O}(1)$	$\mathcal{O}(\log n)^*$	pointer chasing
Pairing heap	$\mathcal{O}(1)$	$\mathcal{O}(\log n)^*$	pointer chasing
Brodal queue	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	huge constants, hard to implement

*amortized

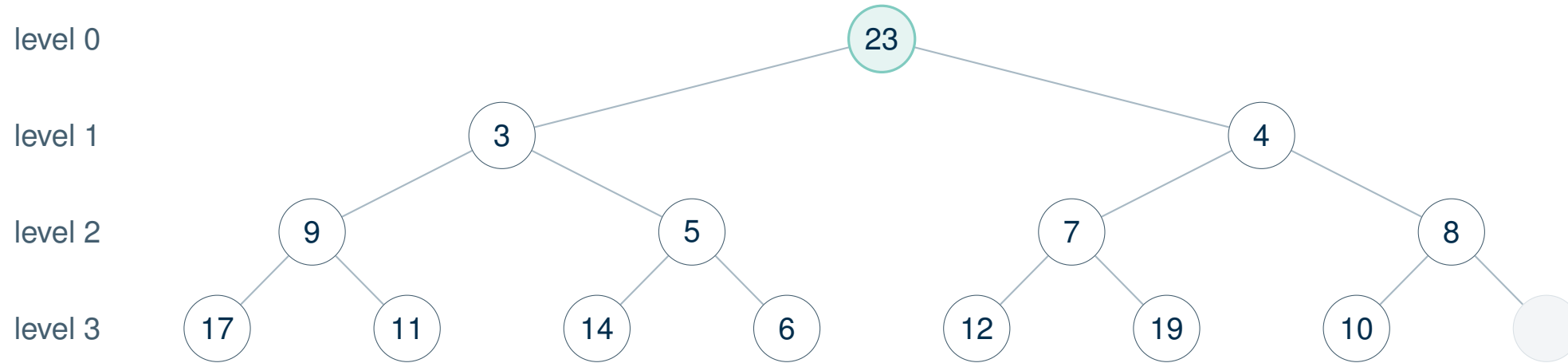
The Binary Heap: pop



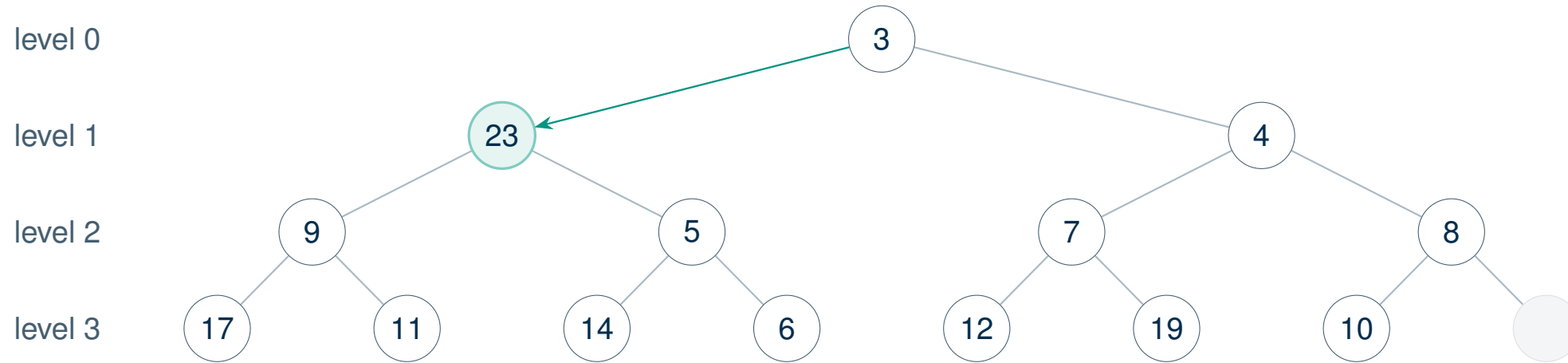
The Binary Heap: pop



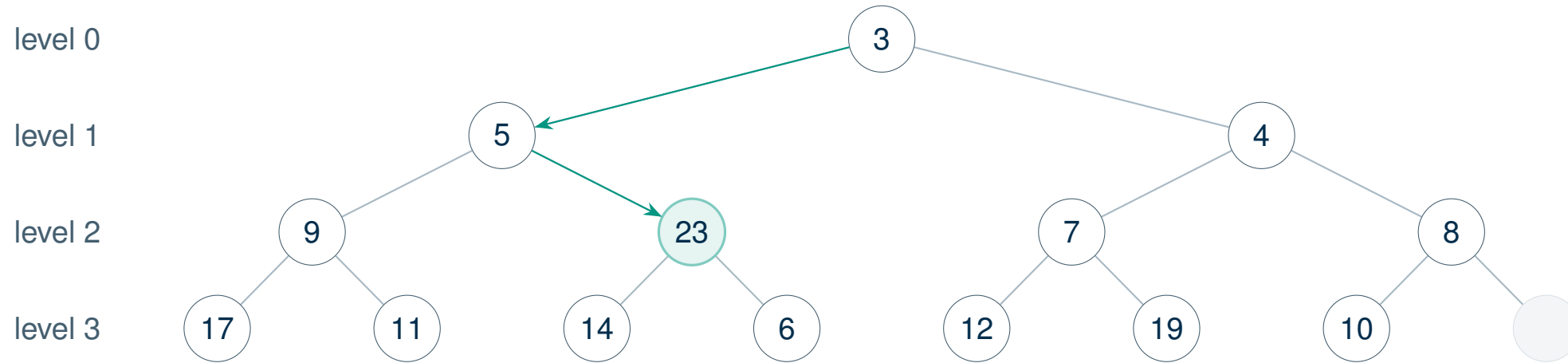
The Binary Heap: pop



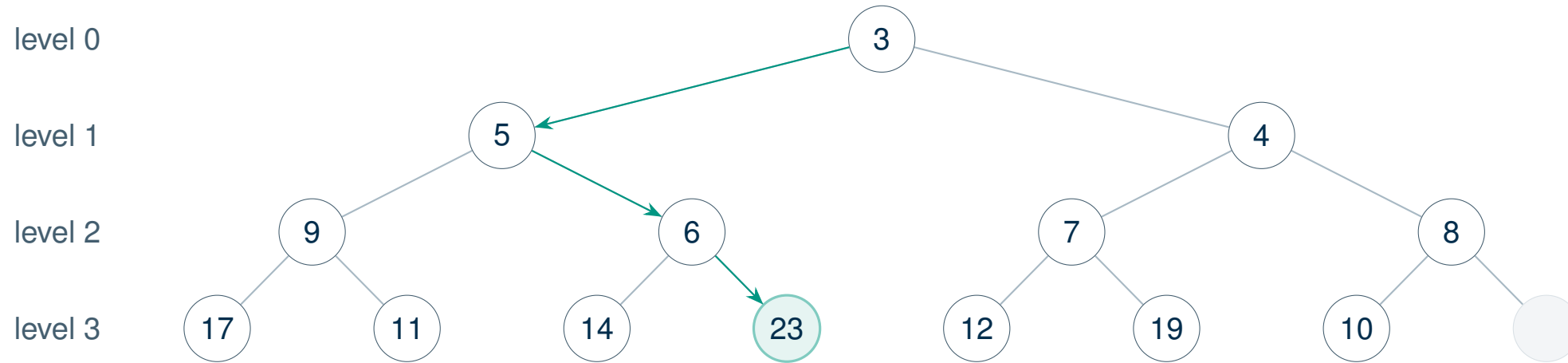
The Binary Heap: pop



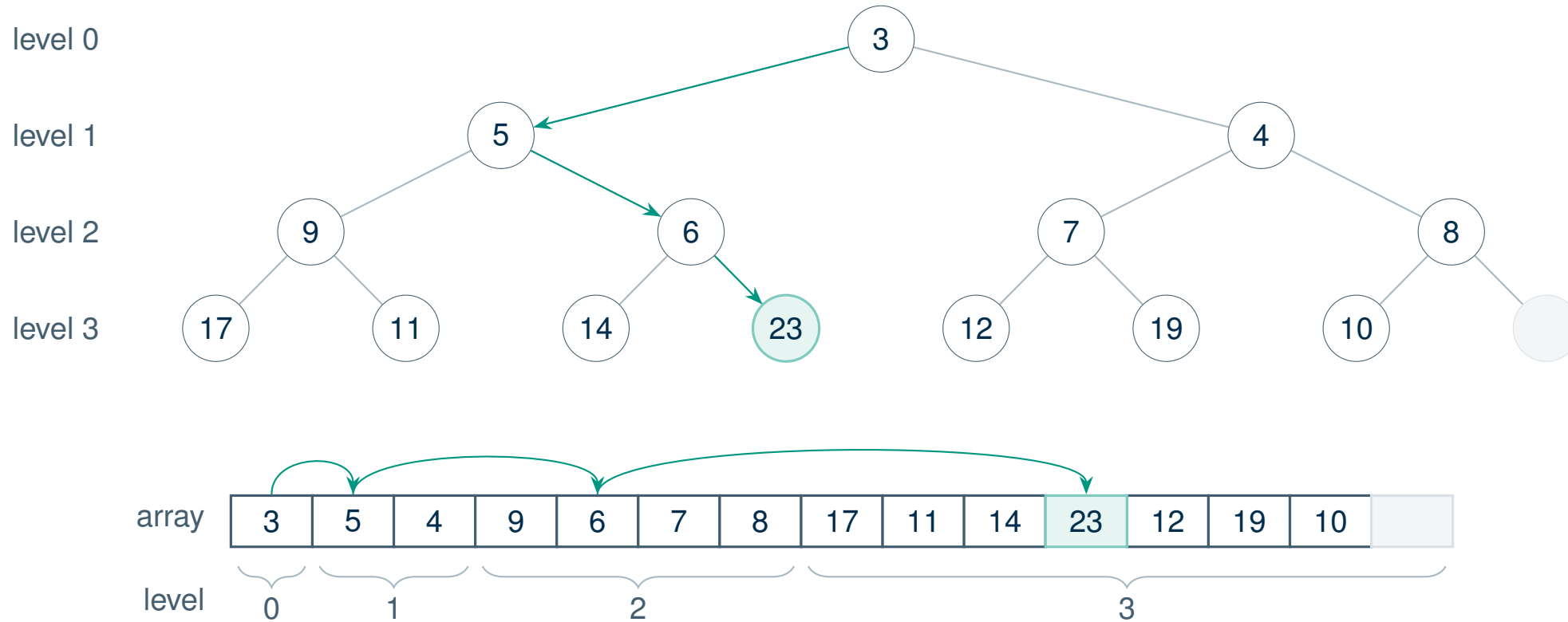
The Binary Heap: pop



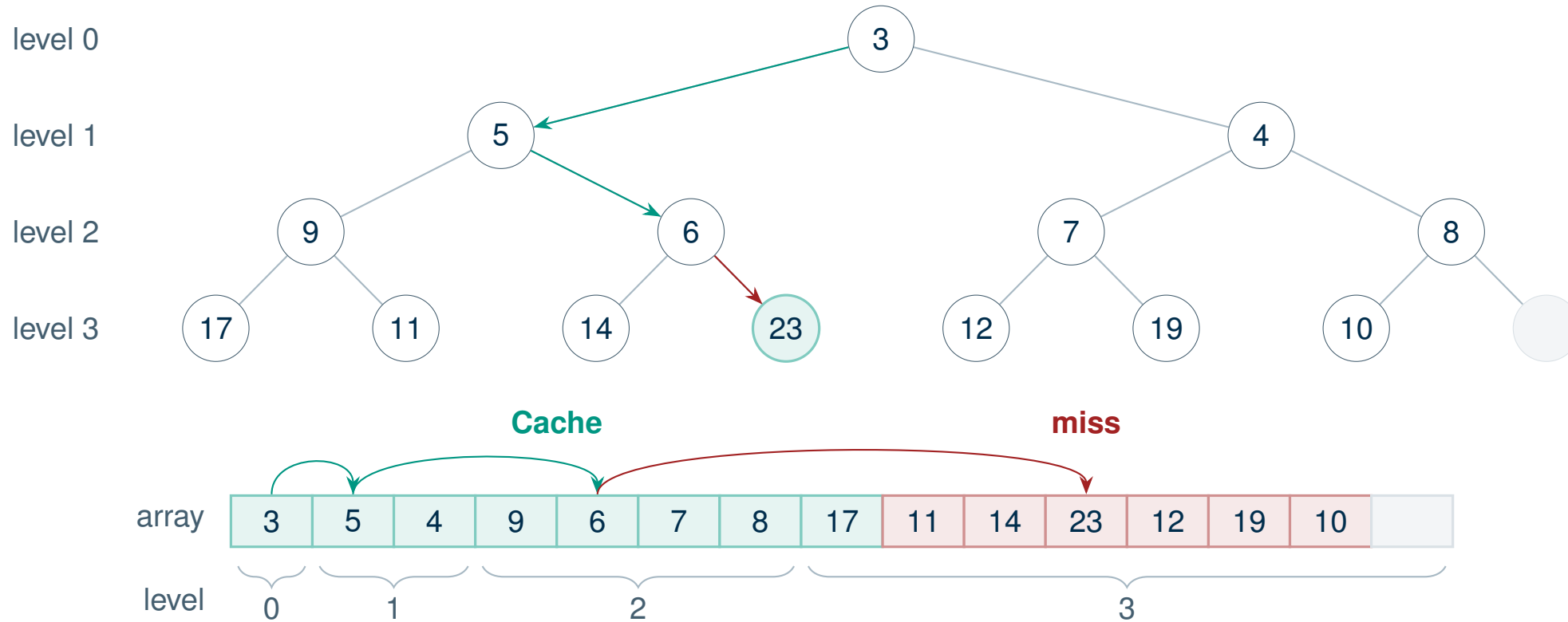
The Binary Heap: pop



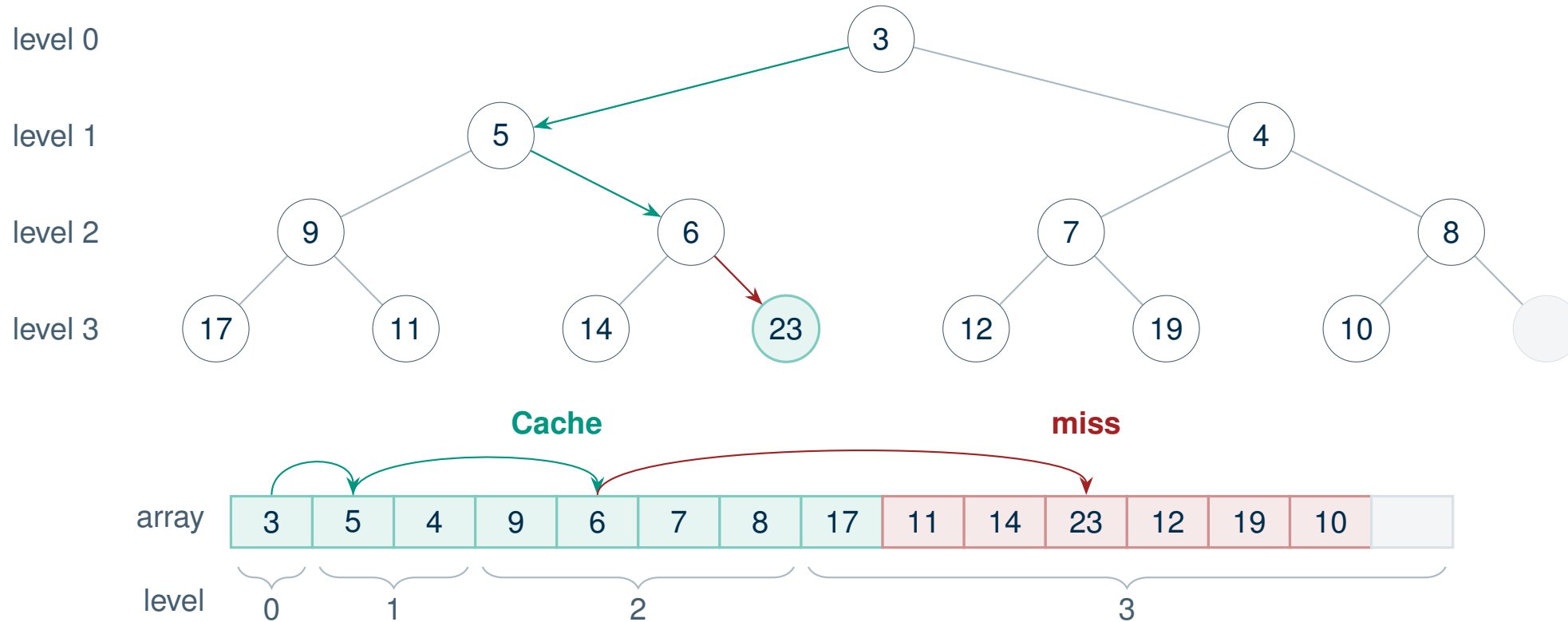
The Binary Heap: pop



The Binary Heap: pop



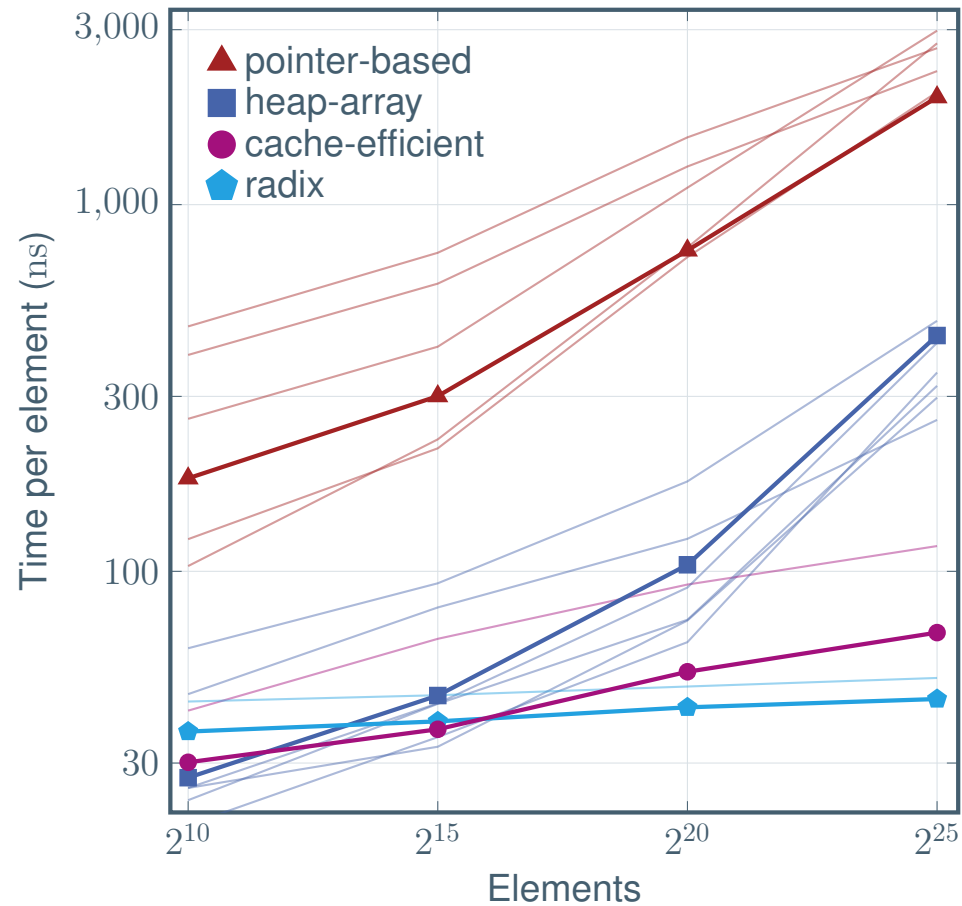
The Binary Heap: pop



Problems: unpredictable branches cache misses inefficient access patterns

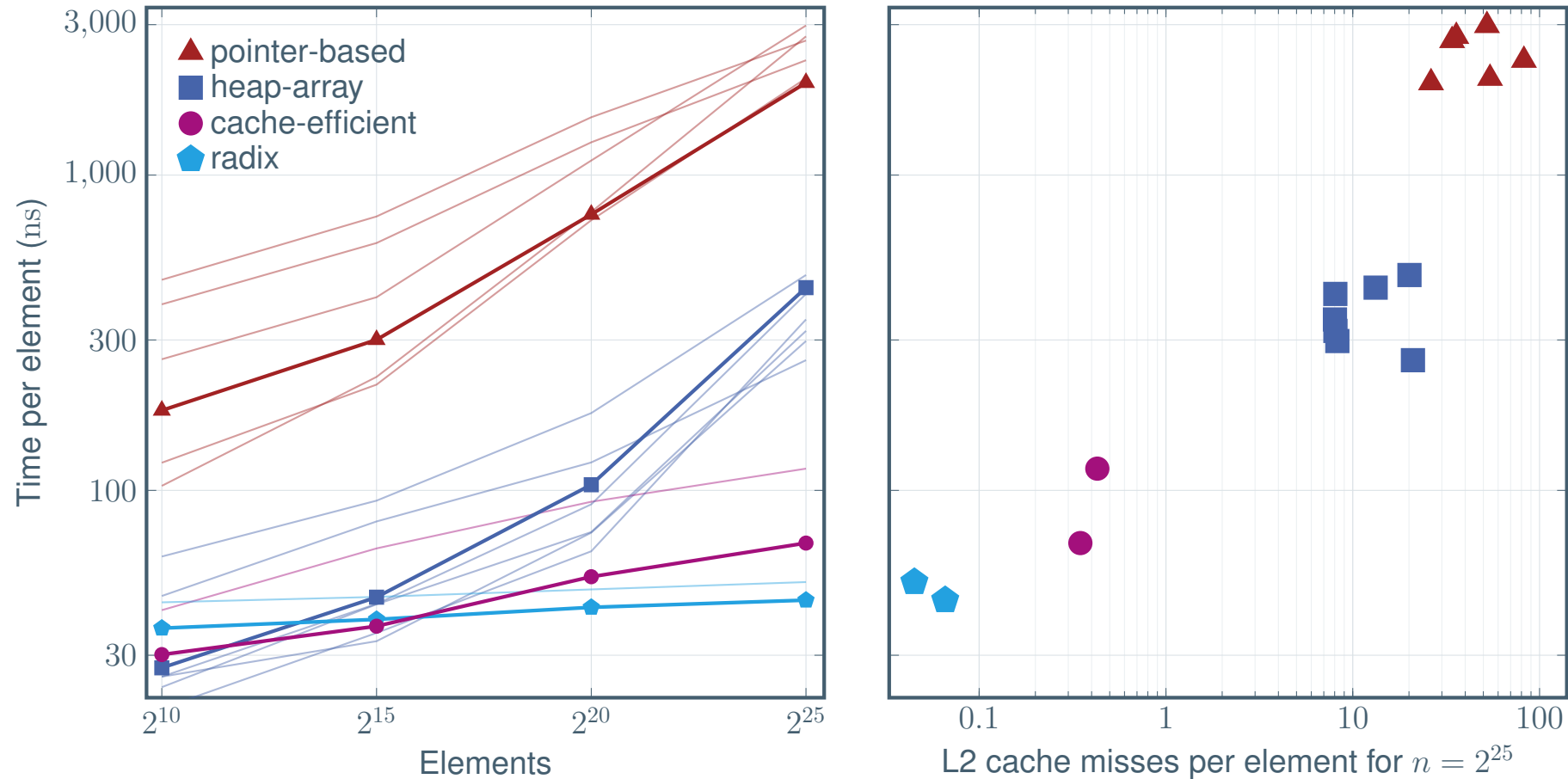
Higher heap arity doesn't help much

Performance vs. Cache Misses



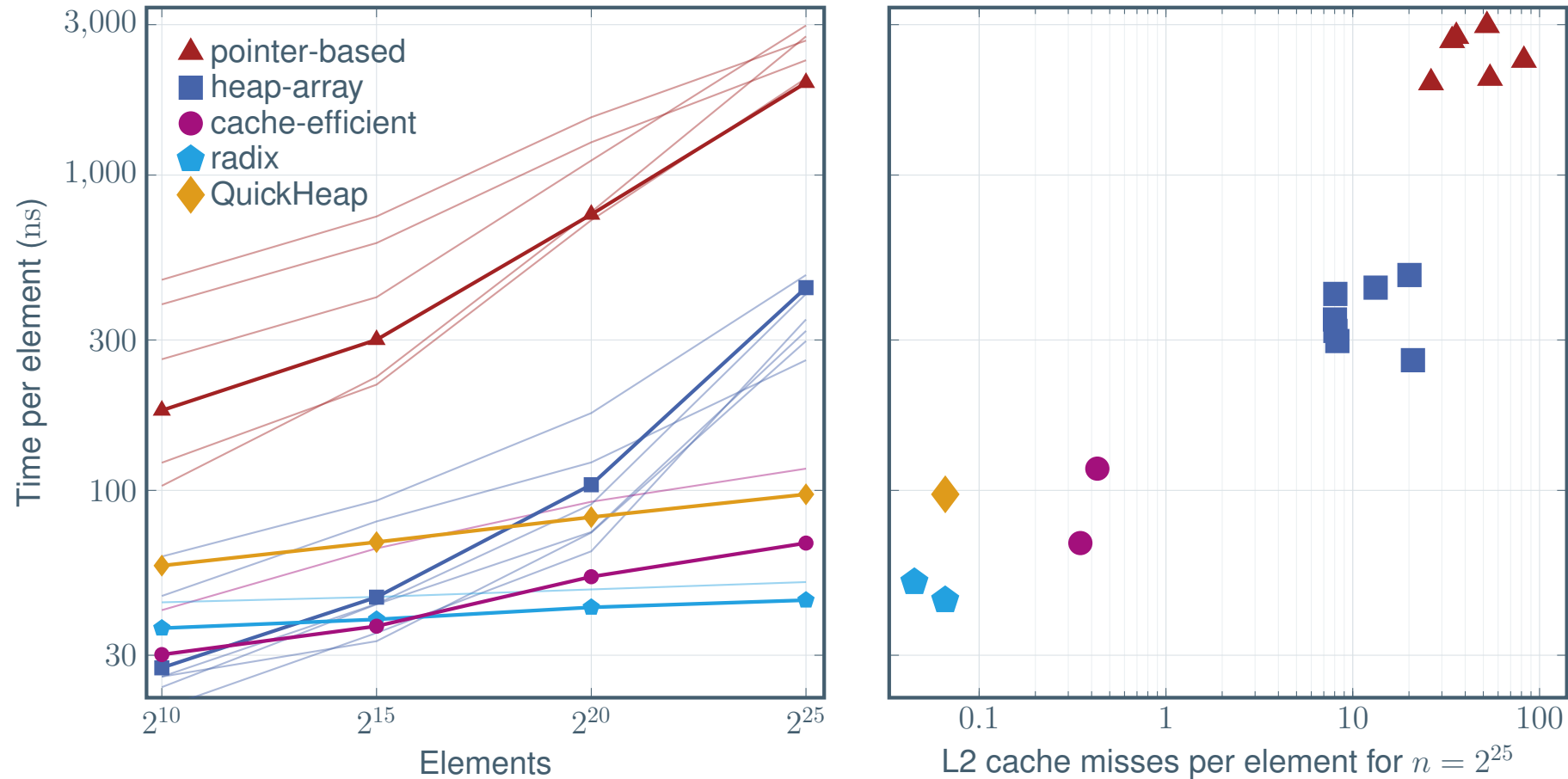
Benchmark: Alternate push and pop at fixed size n with increasing values

Performance vs. Cache Misses



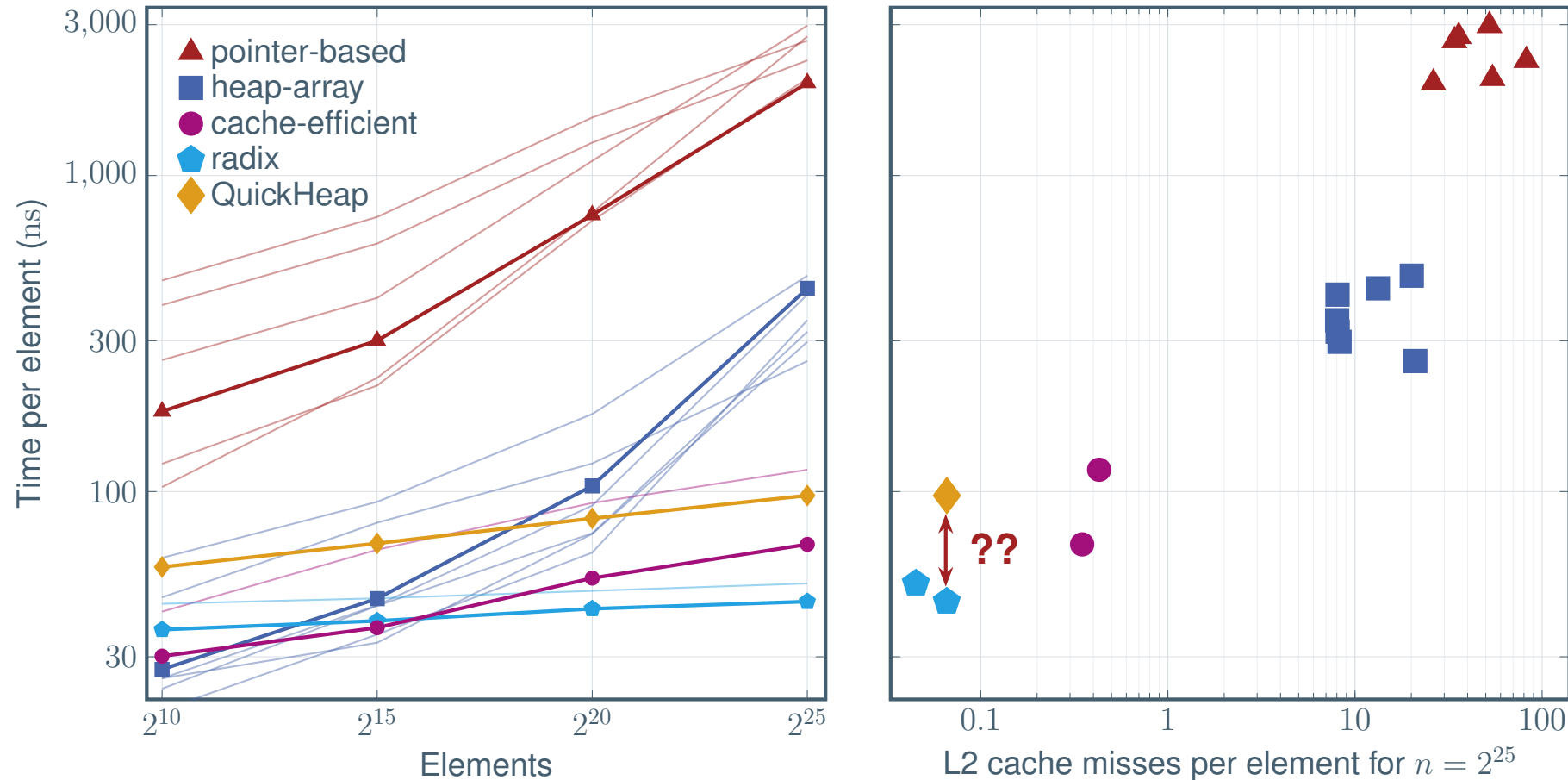
Benchmark: Alternate push and pop at fixed size n with increasing values

Performance vs. Cache Misses



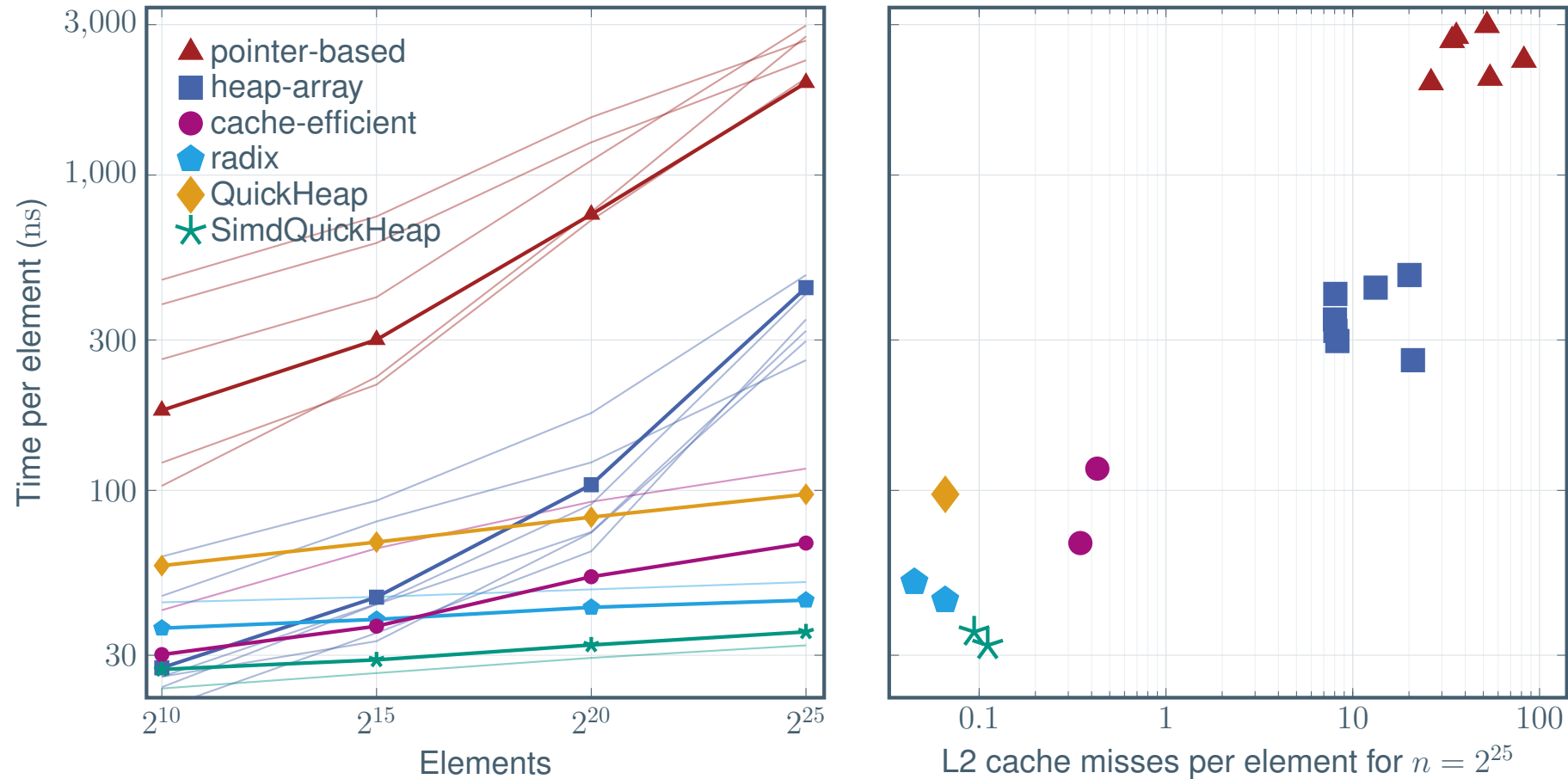
Benchmark: Alternate push and pop at fixed size n with increasing values

Performance vs. Cache Misses



Benchmark: Alternate push and pop at fixed size n with increasing values

Performance vs. Cache Misses



Benchmark: Alternate push and pop at fixed size n with increasing values

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
---	---	---	----	----	---	---	---	---	---	---	----

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
---	---	---	----	----	---	---	---	---	---	---	----

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
---	---	---	----	----	---	---	---	---	---	---	----

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
6	5	3	4	1	2	7	8	13	11	9	10

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
6	5	3	4	1	2	7	8	13	11	9	10

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
6	5	3	4	1	2	7	8	13	11	9	10

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
6	5	3	4	1	2	7	8	13	11	9	10
1	2	3	6	5	4	7	8	13	11	9	10

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
6	5	3	4	1	2	7	8	13	11	9	10
1	2	3	6	5	4	7	8	13	11	9	10

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

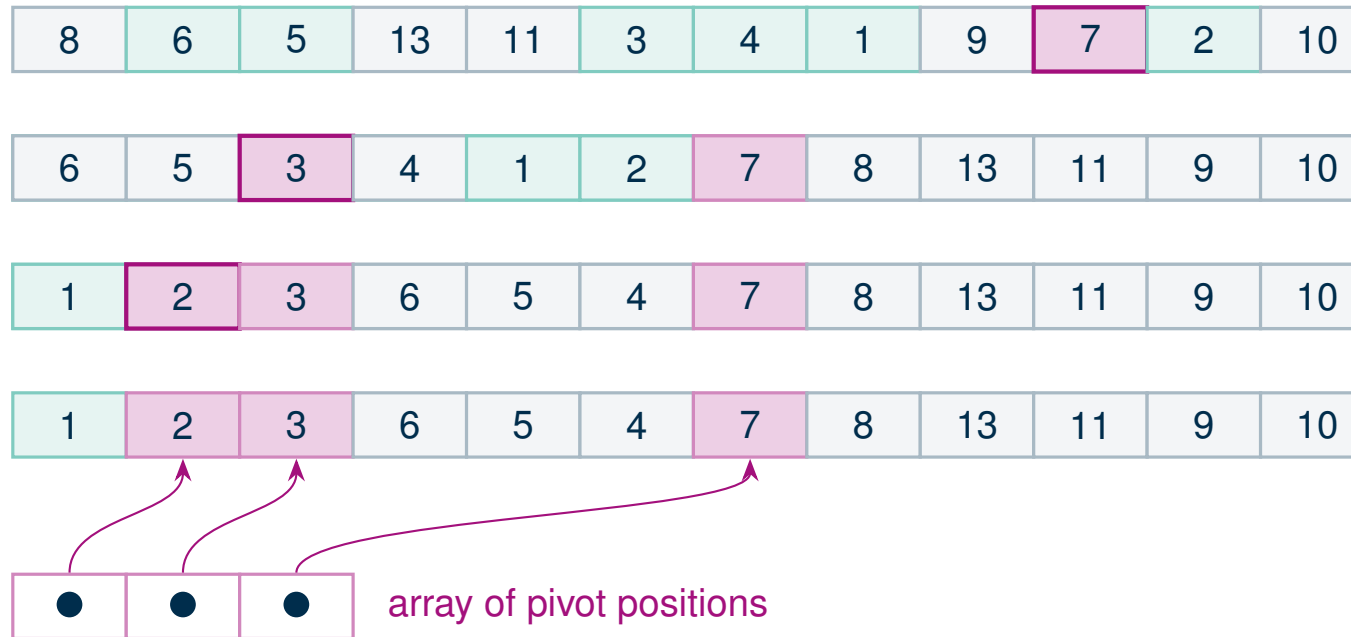
Find the **smallest** element with QuickSelect

8	6	5	13	11	3	4	1	9	7	2	10
6	5	3	4	1	2	7	8	13	11	9	10
1	2	3	6	5	4	7	8	13	11	9	10
1	2	3	6	5	4	7	8	13	11	9	10

The QuickHeap: QuickSelect as a Data Structure

[Navarro & Paredes 2010]

Find the **smallest** element with QuickSelect



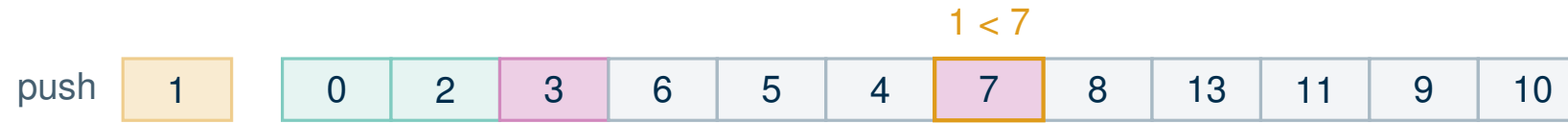
The QuickHeap: push bubbles down

[Navarro & Paredes 2010]



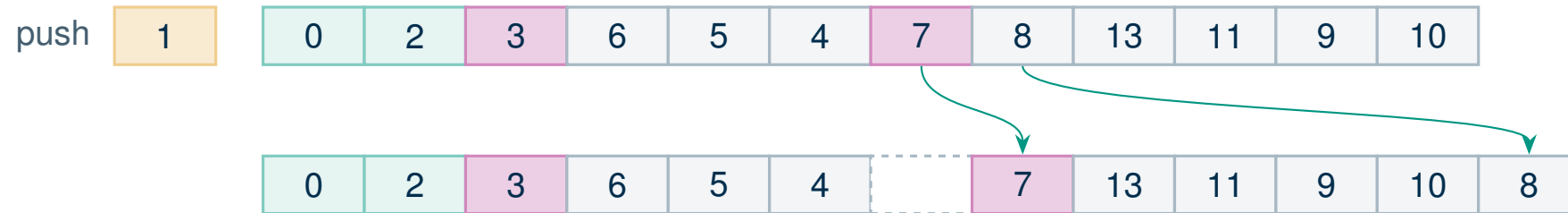
The QuickHeap: push bubbles down

[Navarro & Paredes 2010]



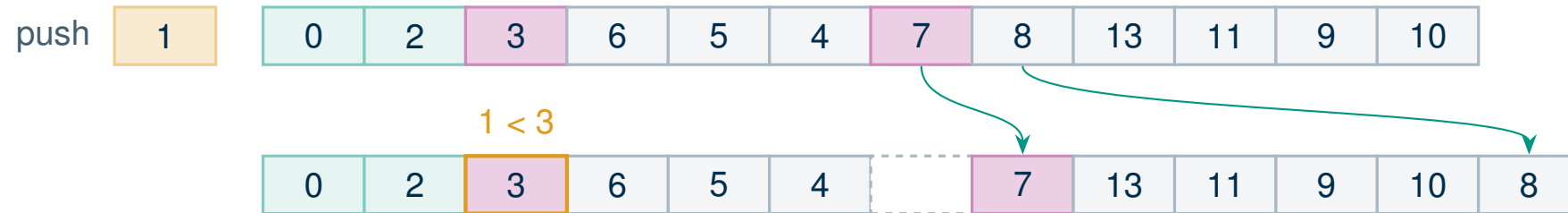
The QuickHeap: push bubbles down

[Navarro & Paredes 2010]



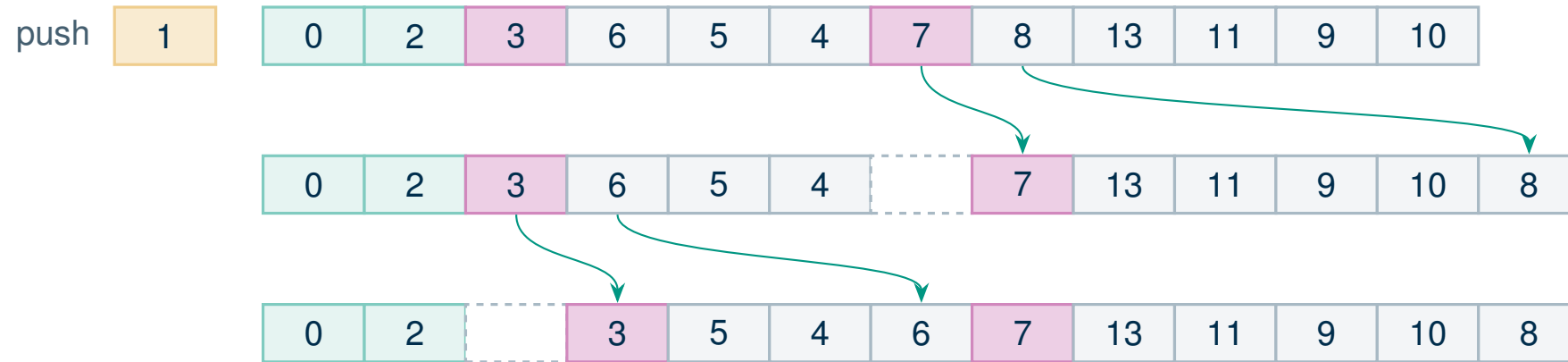
The QuickHeap: push bubbles down

[Navarro & Paredes 2010]



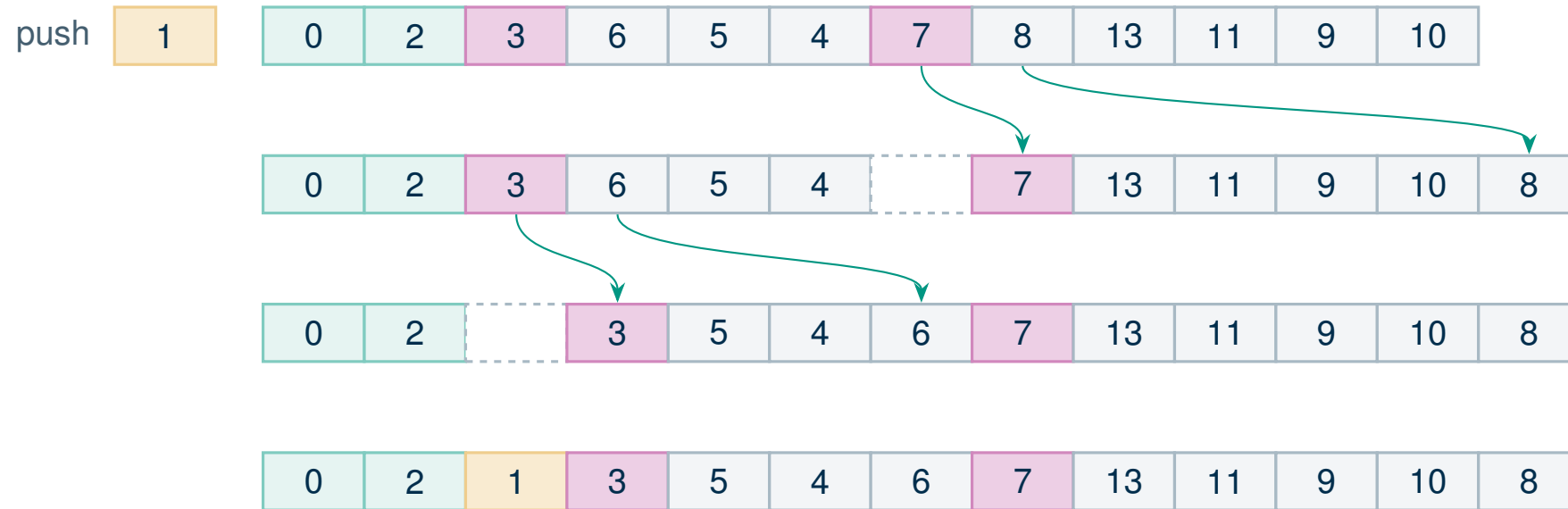
The QuickHeap: push bubbles down

[Navarro & Paredes 2010]



The QuickHeap: push bubbles down

[Navarro & Paredes 2010]



The QuickHeap: pop recursively partitions

[Navarro & Paredes 2010]

0	2	1	3	5	4	6	7	13	11	9	10	8
---	---	---	---	---	---	---	---	----	----	---	----	---

The QuickHeap: pop recursively partitions

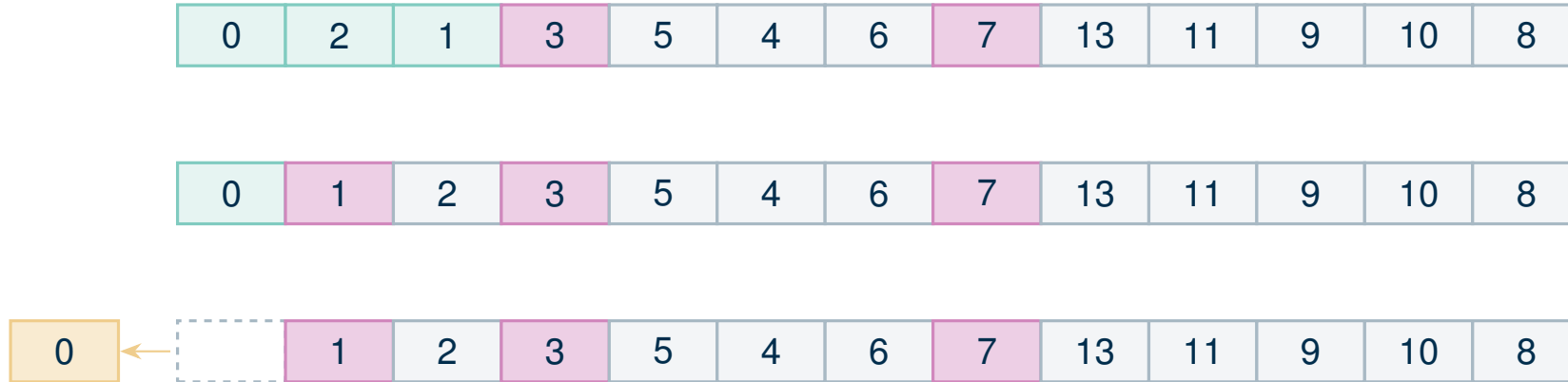
[Navarro & Paredes 2010]

0	2	1	3	5	4	6	7	13	11	9	10	8
---	---	---	---	---	---	---	---	----	----	---	----	---

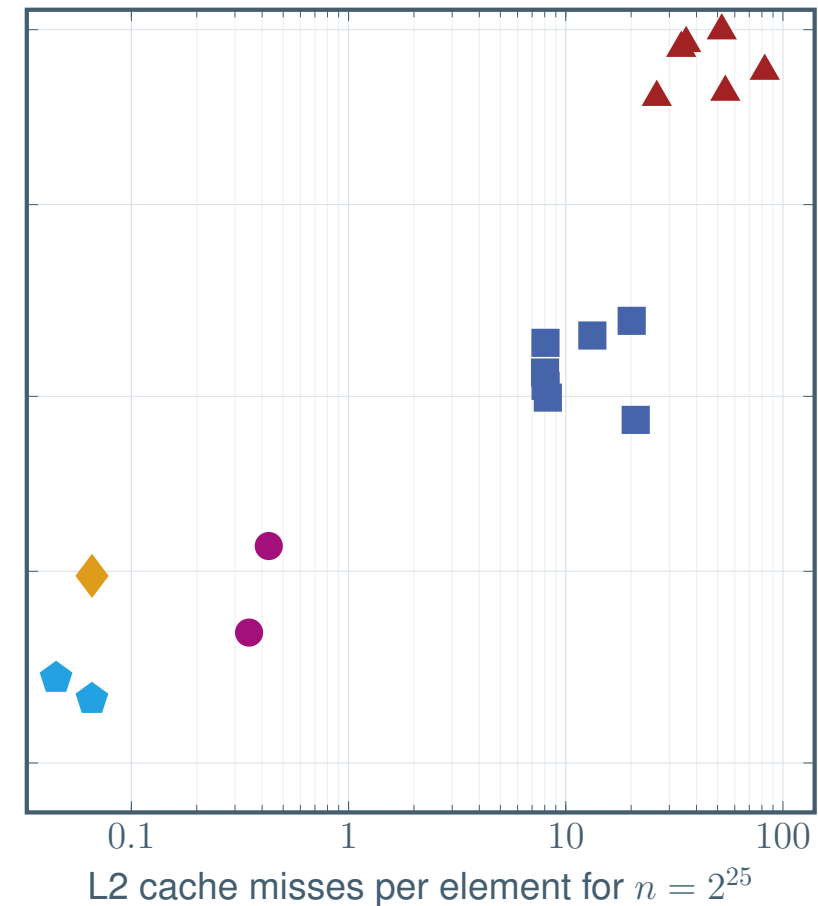
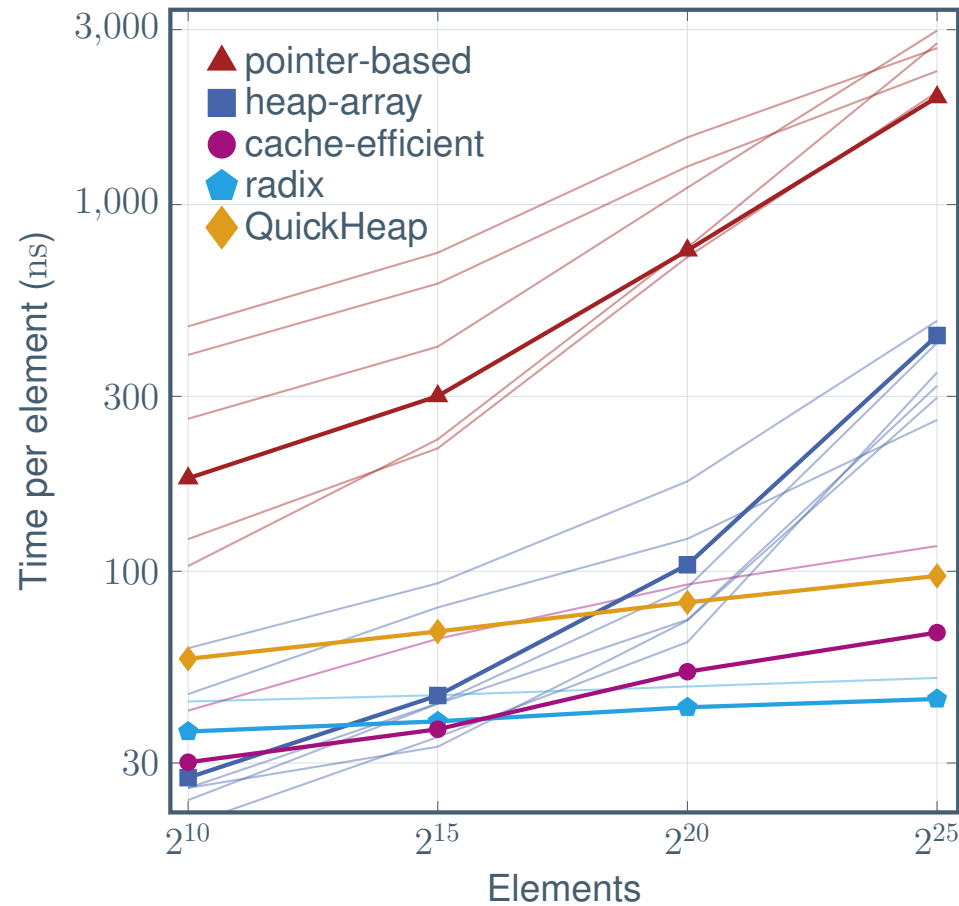
0	1	2	3	5	4	6	7	13	11	9	10	8
---	---	---	---	---	---	---	---	----	----	---	----	---

The QuickHeap: pop recursively partitions

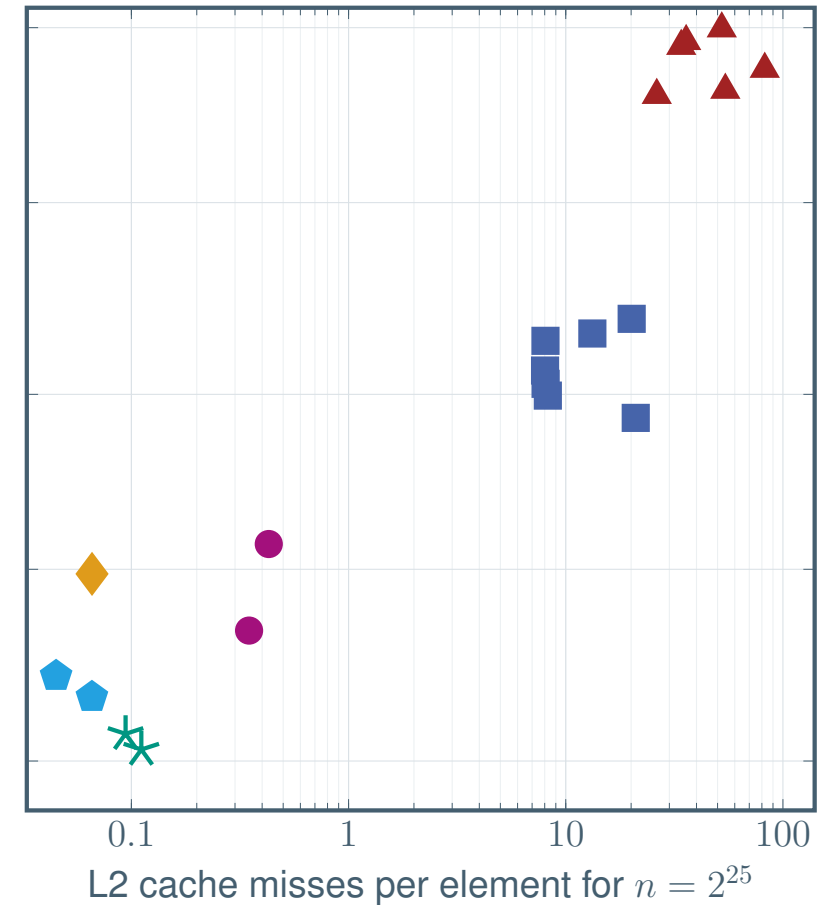
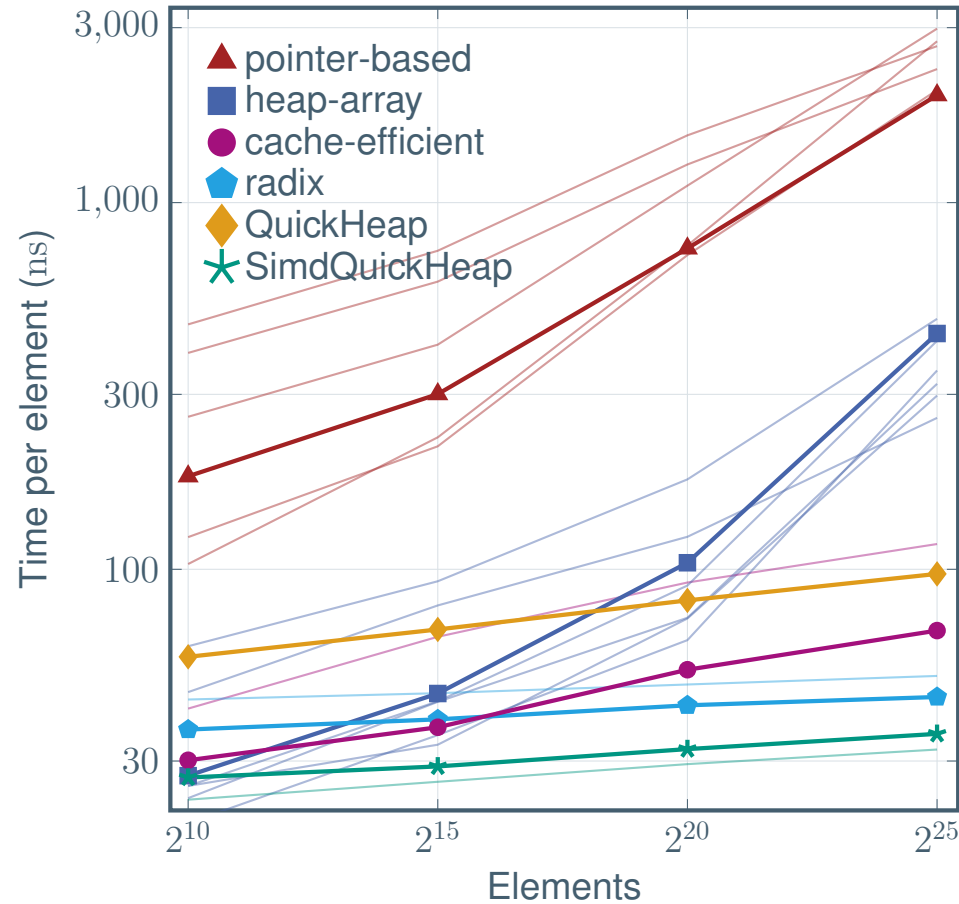
[Navarro & Paredes 2010]



Performance vs. Cache Misses

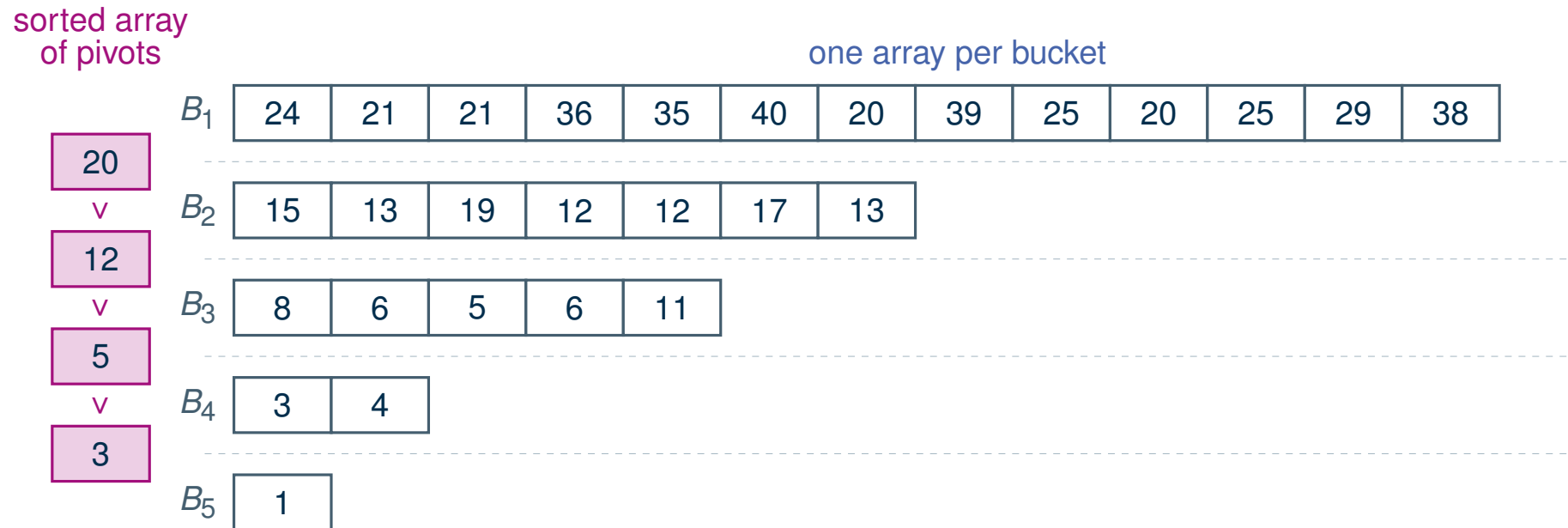


Performance vs. Cache Misses

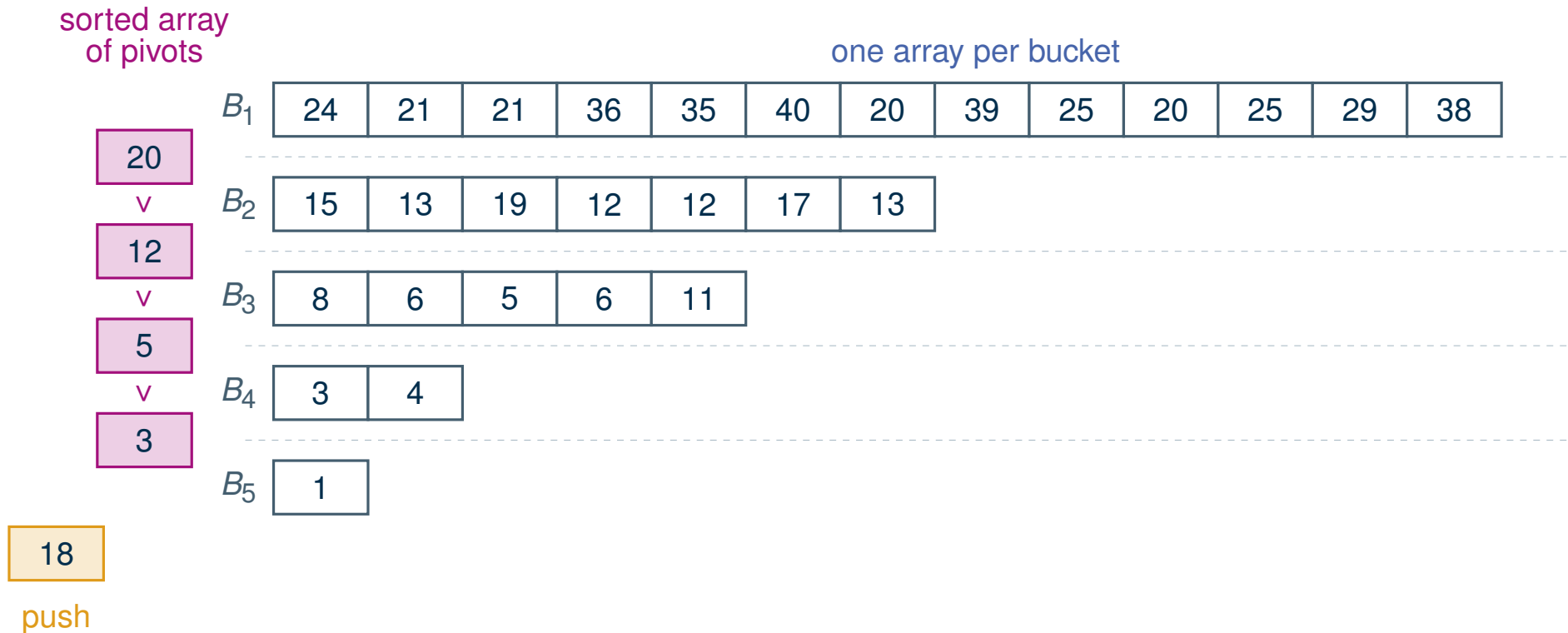


The SimdQuickHeap

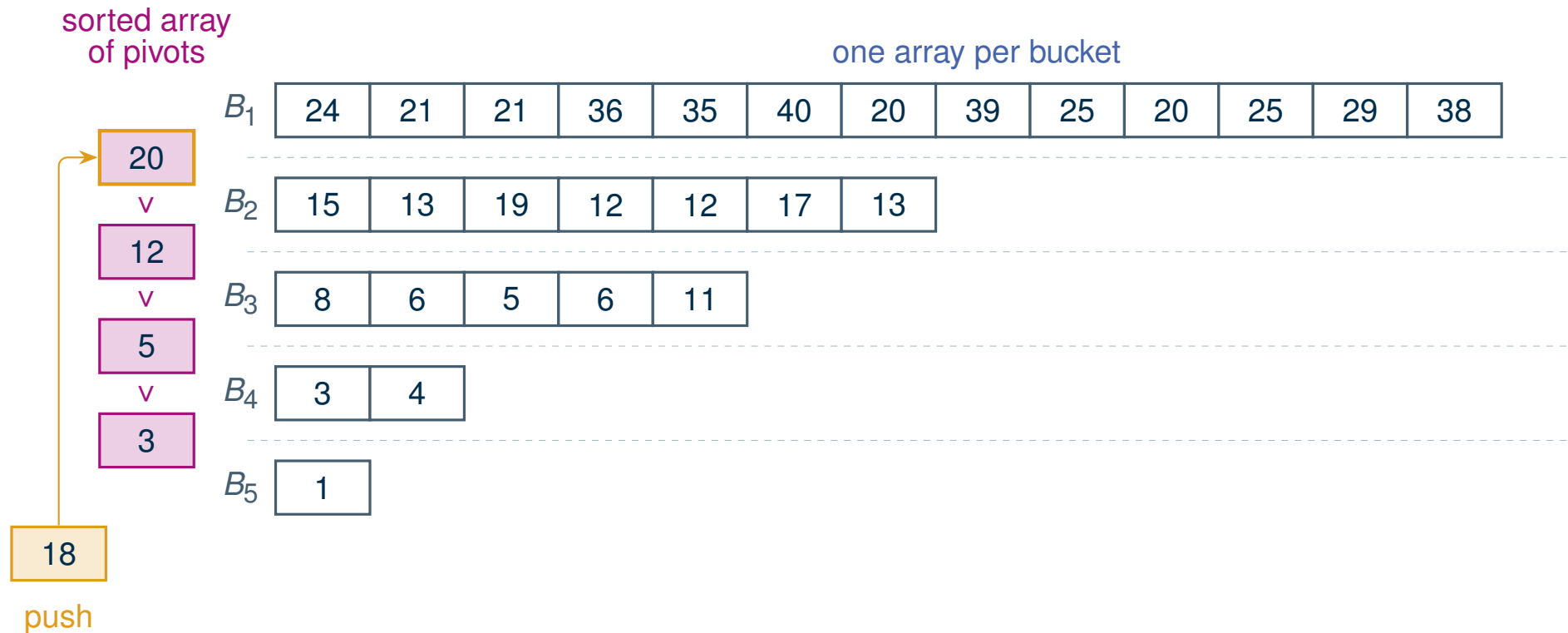
The SimdQuickHeap



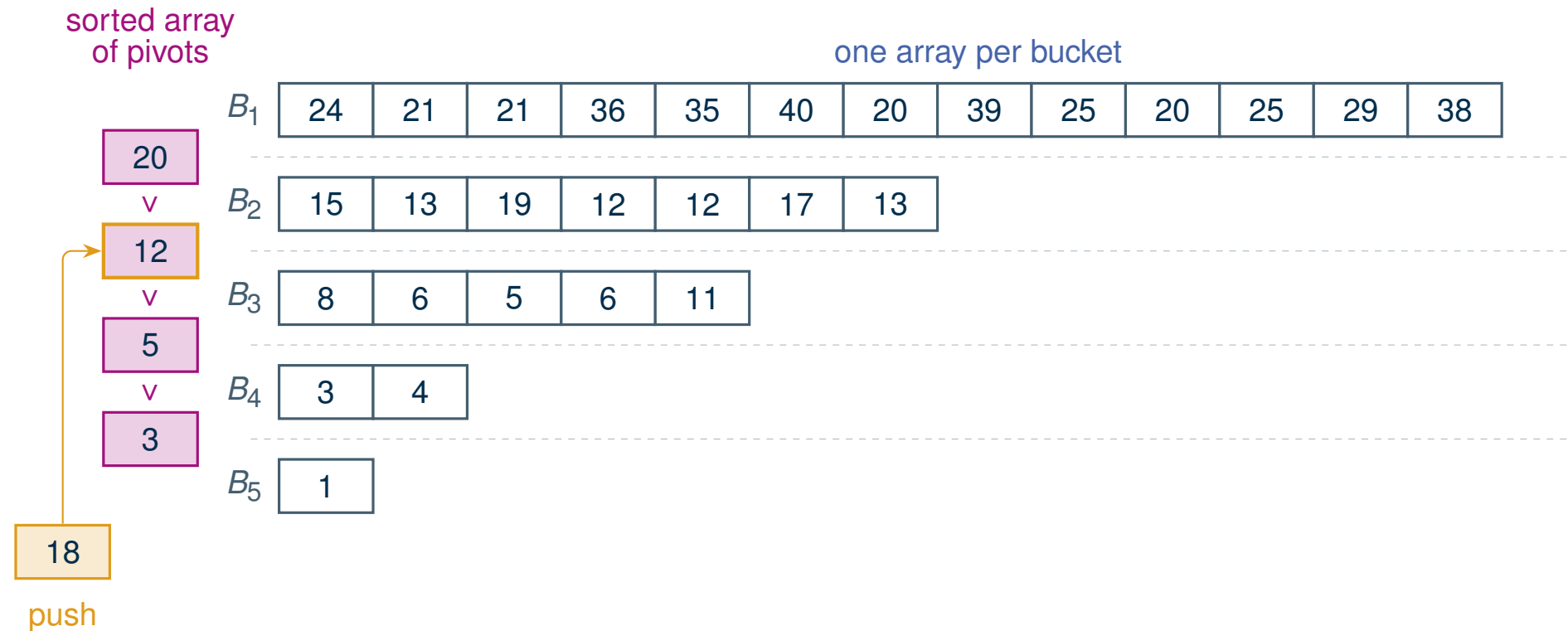
The SimdQuickHeap



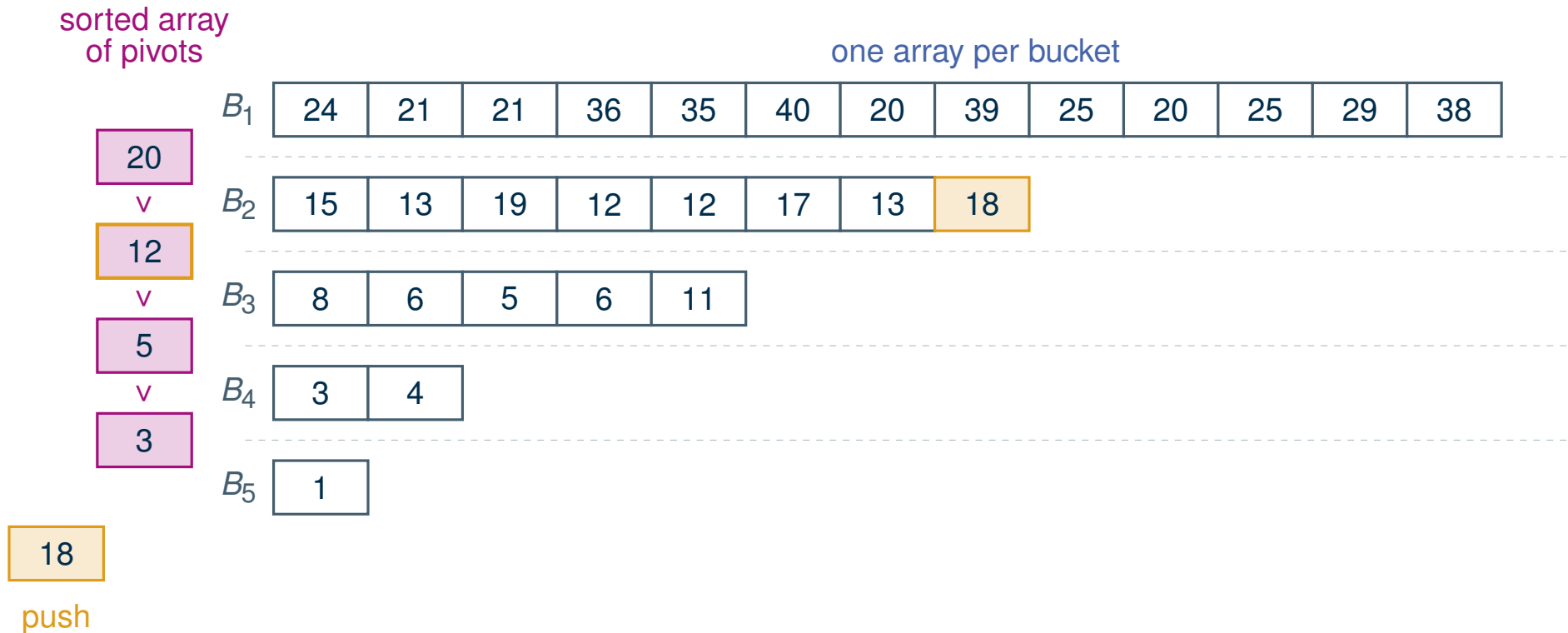
The SimdQuickHeap



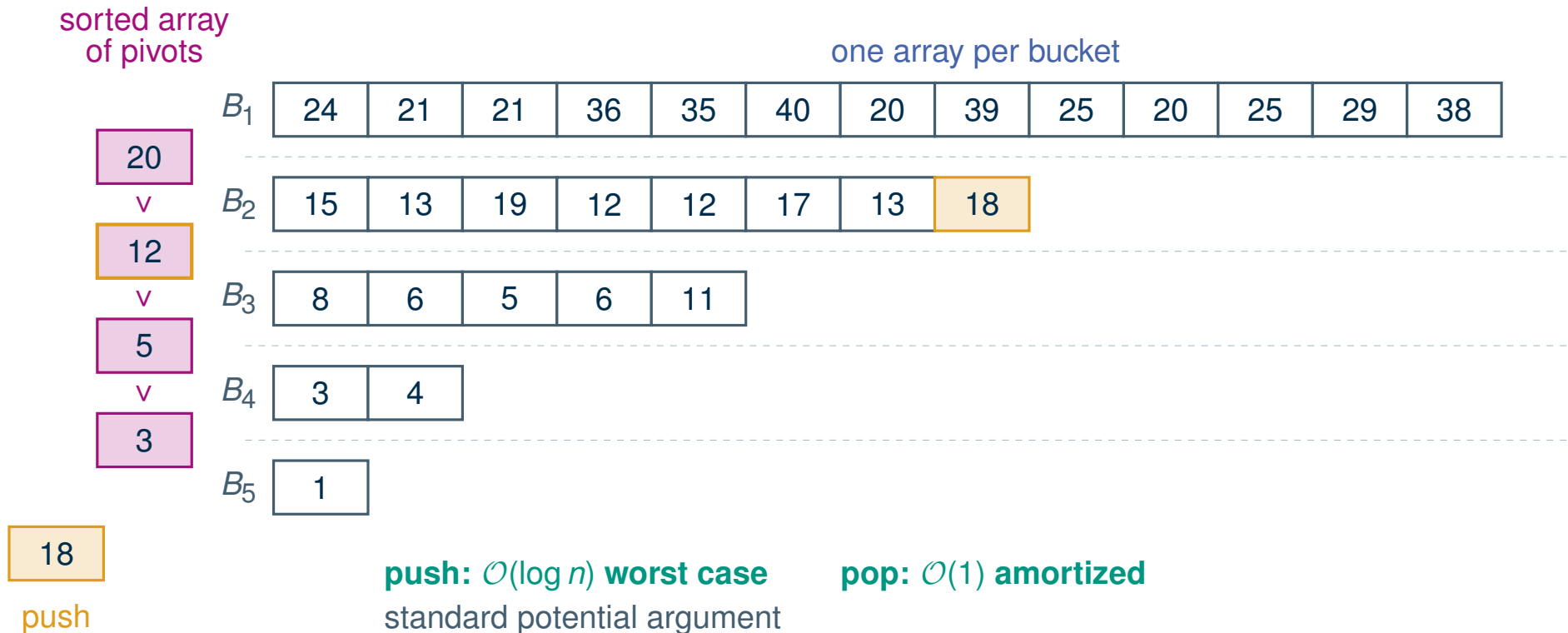
The SimdQuickHeap



The SimdQuickHeap



The SimdQuickHeap



The SimdQuickHeap: Finding the Bucket

broadcast the element

18	18	18	18	18	18	18	18
----	----	----	----	----	----	----	----

The SimdQuickHeap: Finding the Bucket

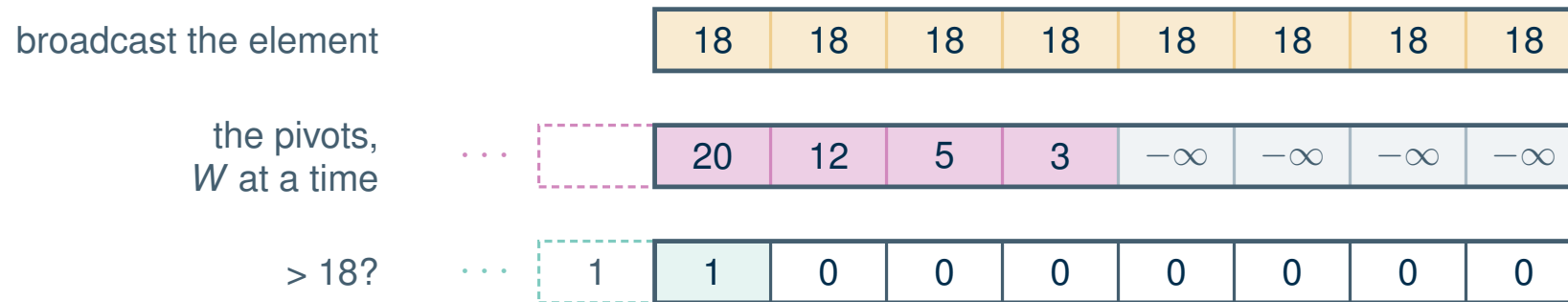
broadcast the element

18	18	18	18	18	18	18	18
----	----	----	----	----	----	----	----

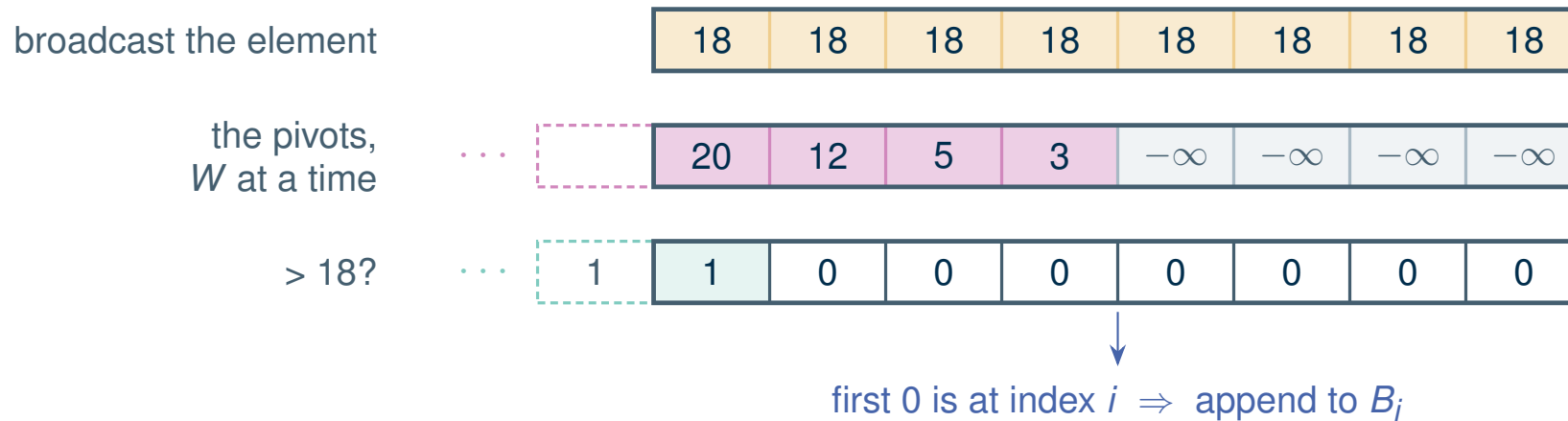
the pivots,
 W at a time

...	20	12	5	3	$-\infty$	$-\infty$	$-\infty$	$-\infty$
-----	----	----	---	---	-----------	-----------	-----------	-----------

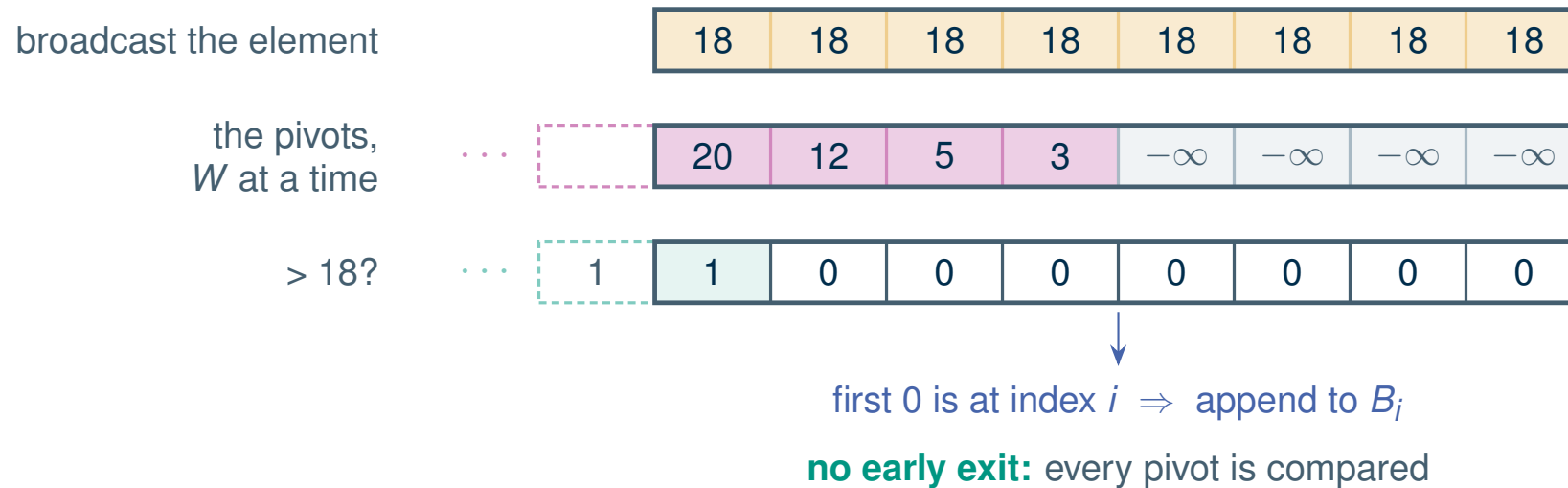
The SimdQuickHeap: Finding the Bucket



The SimdQuickHeap: Finding the Bucket



The SimdQuickHeap: Finding the Bucket



The SimdQuickHeap: Partitioning the Bucket

broadcast pivot

7	7	7	7	7	7	7	7
---	---	---	---	---	---	---	---

The SimdQuickHeap: Partitioning the Bucket

broadcast pivot

7	7	7	7	7	7	7	7
---	---	---	---	---	---	---	---

load B_ℓ ,
 W at a time

...	8	6	5	13	11	3	4	1	...
-----	---	---	---	----	----	---	---	---	-----

The SimdQuickHeap: Partitioning the Bucket

broadcast pivot

7	7	7	7	7	7	7	7
---	---	---	---	---	---	---	---

load B_ℓ ,
 W at a time

...	8	6	5	13	11	3	4	1	...
-----	---	---	---	----	----	---	---	---	-----

< 7?

0	1	1	0	0	1	1	1
---	---	---	---	---	---	---	---

The SimdQuickHeap: Partitioning the Bucket

broadcast pivot

7	7	7	7	7	7	7	7
---	---	---	---	---	---	---	---

load B_ℓ ,
 W at a time

...	8	6	5	13	11	3	4	1	...
-----	---	---	---	----	----	---	---	---	-----

< 7?

0	1	1	0	0	1	1	1
---	---	---	---	---	---	---	---

compress

8	13	11					
6	5	3	4	1			

The SimdQuickHeap: Partitioning the Bucket

broadcast pivot

7	7	7	7	7	7	7	7
---	---	---	---	---	---	---	---

load B_ℓ ,
 W at a time

...	8	6	5	13	11	3	4	1	...
-----	---	---	---	----	----	---	---	---	-----

< 7?

0	1	1	0	0	1	1	1
---	---	---	---	---	---	---	---

compress

8	13	11					
---	----	----	--	--	--	--	--

6	5	3	4	1			
---	---	---	---	---	--	--	--

append to B_ℓ



append to $B_{\ell+1}$

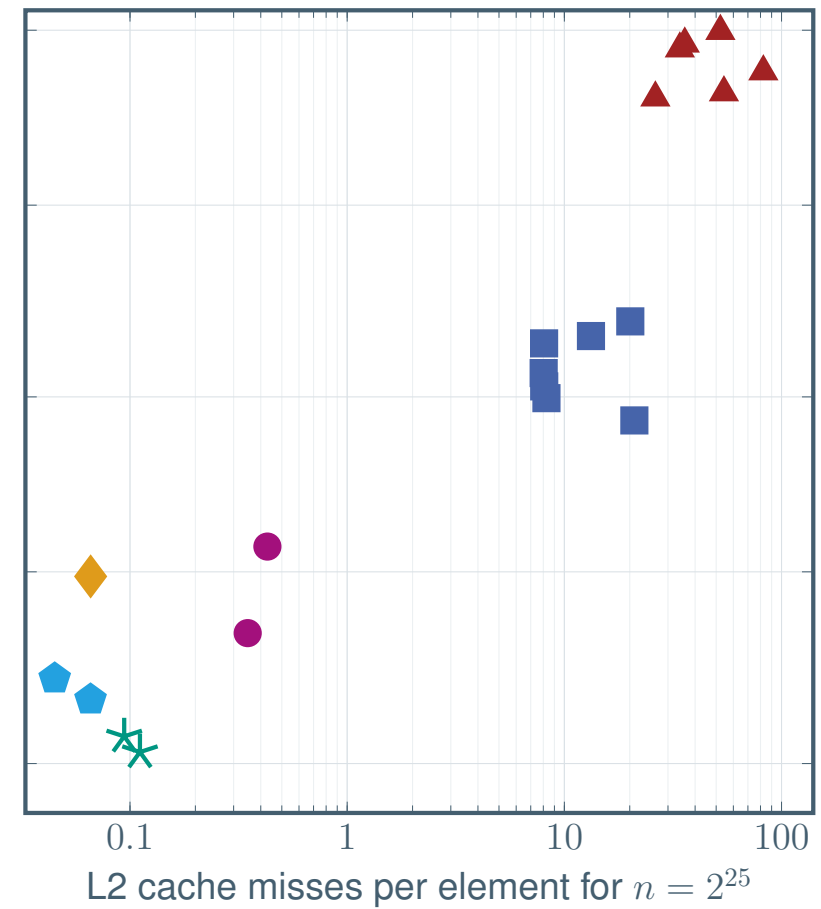
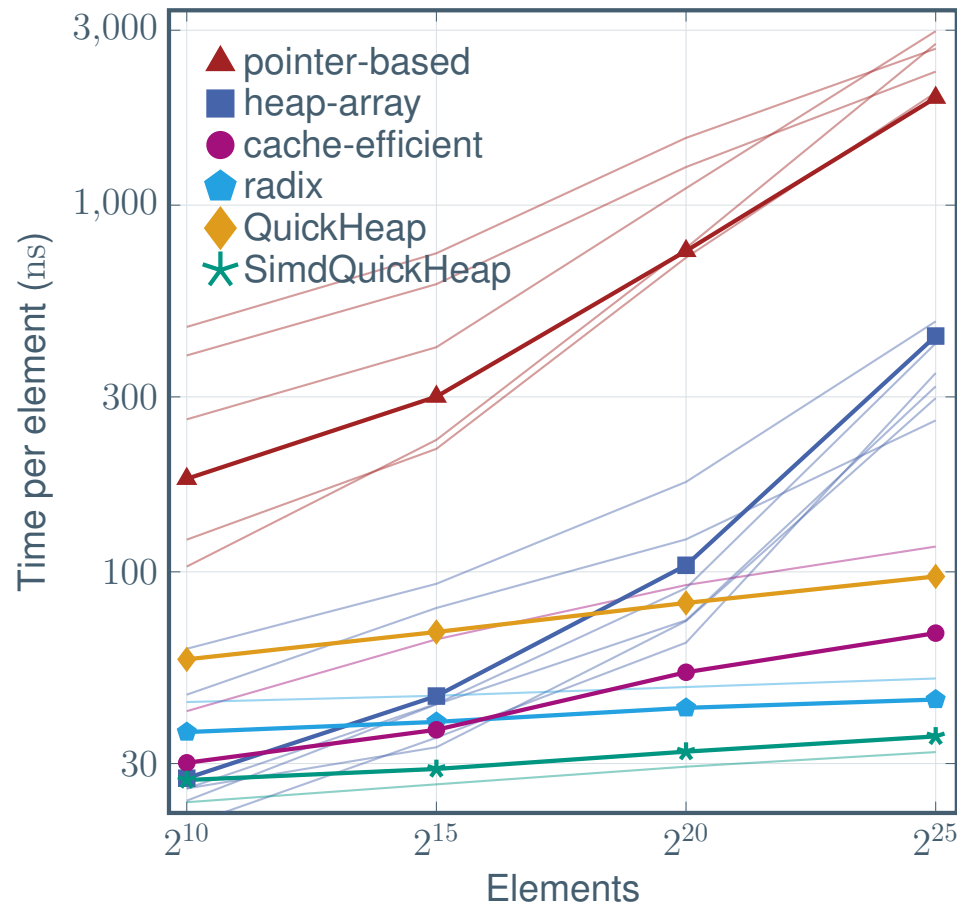


Implementation Details

- **Pivot selection.** Median of three random samples
- **Equal elements.** Use $<$ for elements **left** of the pivot and $\leq$ for elements right of the pivot
- **Small buckets.** Don't partition the last bucket if it contains fewer than 16 elements
- **Classification.** SIMD linear scan up to 64 pivots, binary search fallback

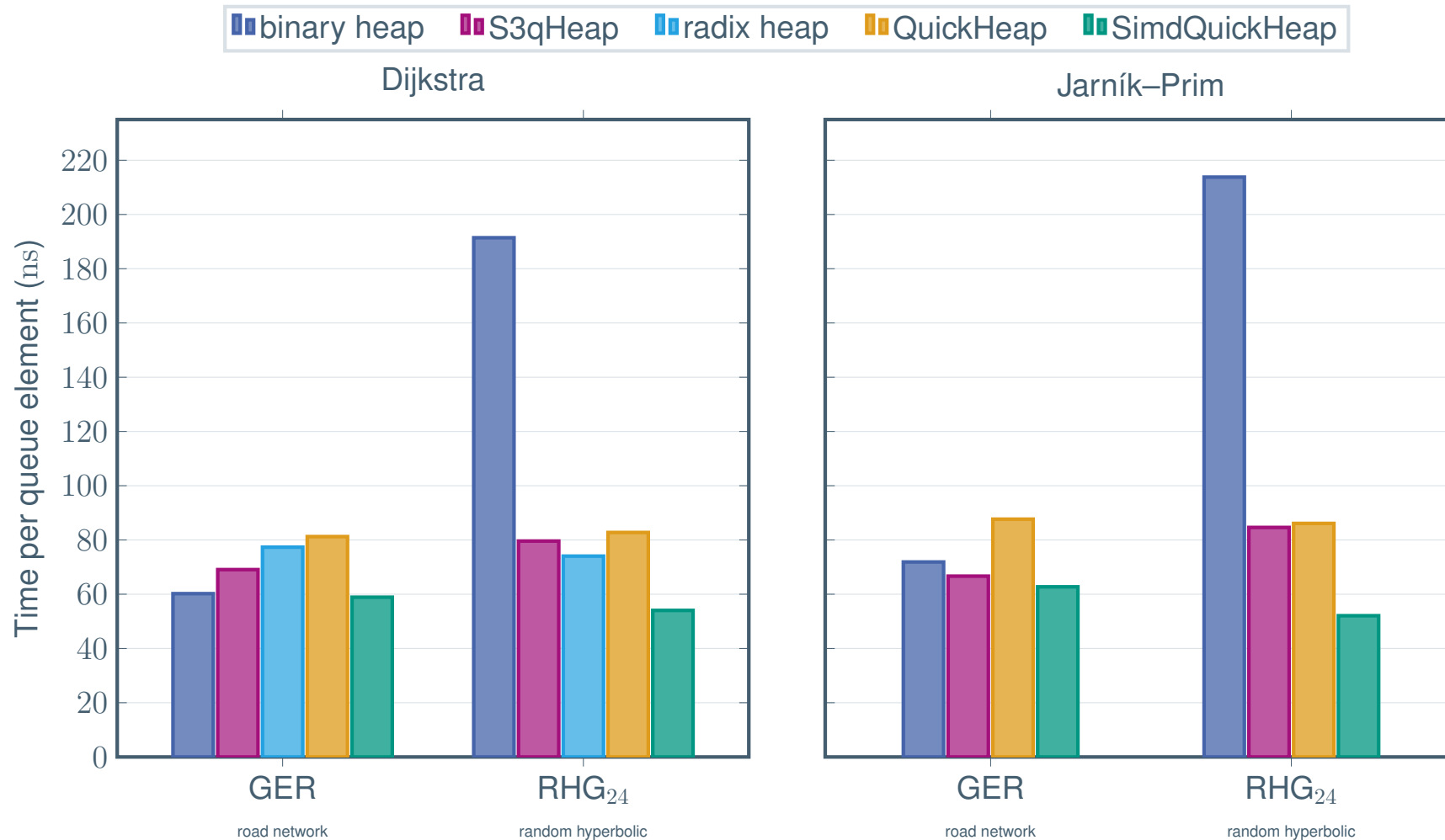
Final speedups

2x S3qHeap, 3x QuickHeap, 10x binary heap



Graph Benchmarks

Dijkstra (SSSP) and Jarník–Prim (MST)

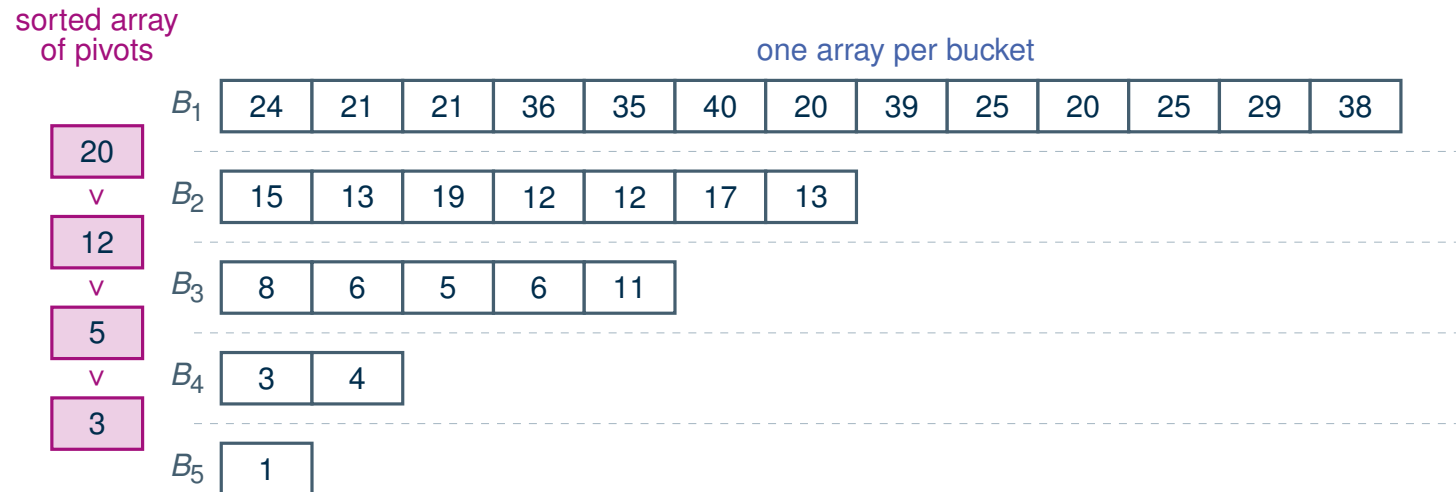


Future Work

- **Data types.** Support (u64, u64) key-value pairs
- **Fix degenerate cases.** Guarantee $\mathcal{O}(\log n)$ layers using rebalancing of *lazy partition heaps* [Brodal et al. 2026]
- **Multi-way splitting.** 4-way partitioning to improve I/O efficiency
- **Memory.** Use a list of blocks to reduce allocations

Conclusion

- Memory access is the main bottleneck for PQs
- Memory access patterns matter
- Neither RAM nor I/O model predict performance
- SIMD works better for **splitting** than for **merging**
- SimdQuickHeap is the fastest PQ in (almost) every benchmark
- The PQ was never the bottleneck for SSSP on road networks



P99 CONF

github.com/RagnarGrootKoerkamp/quickheap
curiouscoding.nl/posts/quickheap

Thank you! Let's connect.



Ragnar Groot Koerkamp
ragnar.grootkoerkamp@gmail.com
[@curiouscoding.nl](https://curiouscoding.nl)



Johannes Breitling
johannes.breitling@student.kit.edu



Marvin Williams
marvin.williams@kit.edu



Backup: SIMD Improvement

